

PyLab – Ein Framework zur Regelungstechnik mit graphenbasierter Verifikation von Regelkreisen

Stephan Epp

4. März 2026

Inhaltsverzeichnis

1	Einführung	2
1.1	Zielsetzung	2
1.2	Aufbau der Arbeit	2
2	Grundlagen	2
2.1	Regelungstechnische Grundbegriffe	2
2.2	Signalflussgraph eines Regelkreises	3
2.3	Graphentheoretische Grundbegriffe	3
3	Softwarearchitektur von PyLab	4
3.1	Modulübersicht	4
3.2	Paketdiagramm	4
3.3	Klassendiagramm	4
3.4	Kernklassen	7
3.4.1	TransferFunction	7
3.4.2	StateSpace	7
3.4.3	PIDController / DiscretePIDController	7
4	Regelkreis-Verifikation	7
4.1	Grundidee	7
4.2	Verarbeitungspipeline	7
4.3	Der Subgraph Algorithmus	8
4.3.1	Signaturen	8
4.3.2	Zyklische Rotation	8
4.3.3	Subgraph-Kriterium	9
4.4	Entscheidungen des Algorithmus	9
4.5	Implementierung: <code>verification.py</code>	9
4.6	Datenklasse <code>VerificationResult</code>	10
4.7	Padding unterschiedlich großer Graphen	11
5	Biologisch orientierte Äquivalenzklassen von Regelkreisen	11
5.1	Motivation und Erkenntnisziel	11
5.2	Grundlegende Konzepte der biologischen Regelung	12
5.2.1	Hierarchische Regelkreisstruktur in Lebewesen	12

5.2.2	Gemeinsame Eigenschaften biologischer Regelkreise	12
5.3	Definition der biologischen Äquivalenzklassen	13
5.4	Klasse I – Der Seestern: Dezentrale Proportionalregelung	13
5.4.1	Biologisches Vorbild: Seestern (<i>Asteroidea</i>)	13
5.4.2	Graphenstruktur der Seestern-Klasse	14
5.4.3	Abgeleitete Entwurfsstrategien	14
5.4.4	Erweiterungspotenzial aus der biologischen Analogie	14
5.5	Klasse II – Pupille: Integrierender Regelkreis mit Sättigung	14
5.5.1	Biologisches Vorbild: Pupillenreflex des Auges	14
5.5.2	Graphenstruktur der Pupillen-Klasse	15
5.5.3	Abgeleitete Entwurfsstrategien	15
5.5.4	Erweiterungspotenzial: Adaptiver PI-Regler	16
5.6	Klasse III – Das Kleinhirn: Vorsteuerung und Lernender Regler	16
5.6.1	Biologisches Vorbild: Kleinhirn (<i>Cerebellum</i>)	16
5.6.2	Graphenstruktur der Kleinhirn-Klasse	17
5.6.3	Abgeleitete Entwurfsstrategien	17
5.6.4	Erweiterungspotenzial: Biologisch motiviertes ILC-Modul	17
5.7	Klasse IV – Das Herz: Autonomer Taktgeber und Kaskadenregelung	18
5.7.1	Biologisches Vorbild: Herzkreislaufsystem	18
5.7.2	Graphenstruktur der Herz-Klasse	18
5.7.3	Abgeleitete Entwurfsstrategien	19
5.7.4	Erweiterungspotenzial: Pulsweitenmodulierter Antrieb	19
5.8	Klasse V – Der Albatros: Energieoptimale Regelung mit Windausnutzung	19
5.8.1	Biologisches Vorbild: Albatros (<i>Diomedea</i>)	19
5.8.2	Graphenstruktur der Albatros-Klasse	20
5.8.3	Abgeleitete Entwurfsstrategien	20
5.8.4	Erweiterungspotenzial: MPC-Modul für PyLab	21
5.9	Klasse VI – Die Mimose: Nichtlinearer Regelkreis mit Hysterese	21
5.9.1	Biologisches Vorbild: Mimose (<i>Mimosa pudica</i>)	21
5.9.2	Graphenstruktur der Mimosen-Klasse	22
5.9.3	Abgeleitete Entwurfsstrategien	22
5.10	Klasse VII – Der Tintenfisch: Multivariable adaptive Regelung	22
5.10.1	Biologisches Vorbild: Tintenfisch (<i>Octopus vulgaris</i>)	22
5.10.2	Graphenstruktur der Tintenfisch-Klasse	23
5.10.3	Abgeleitete Entwurfsstrategien	23
5.10.4	Erweiterungspotenzial: MIMO-Erweiterung von PyLab	24
5.11	Hierarchie der biologischen Äquivalenzklassen	24
5.12	Klassifikationsalgorithmus	25
5.12.1	Referenztopologien	25
5.12.2	Klassifikationsablauf	25
5.12.3	Implementierung: <code>bio_classify.py</code>	26
5.13	Anwendungsbeispiele der biologischen Klassifikation	26
5.13.1	Beispiel 1: Temperaturregelung einer Heizung (Klasse II, Pupille)	26
5.13.2	Beispiel 2: Roboterarm-Positionsregelung (Klasse III, Kleinhirn)	27
5.13.3	Beispiel 3: Windkraftanlage (Klasse V, Albatros)	27
5.14	Zusammenfassung und Ausblick	27

6	GUI-Integration	28
6.1	Verifikations-Dock	28
6.2	Automatische Synchronisation	28
6.3	Fehlerbehandlung in der GUI	28
7	Analyse	29
7.1	Zeitkomplexität	29
7.2	Speicherkomplexität	29
7.3	Korrektheit	29
7.4	Adjazenzlisten-Variante	29
7.5	Testabdeckung	29
8	Anwendungsbeispiele	29
8.1	Beispiel 1: P-Regler vs. PI-Regler	29
8.2	Beispiel 2: Strukturell verschiedene Regelkreise	30
8.3	Beispiel 3: Äquivalente Regelkreise	31
9	Zusammenfassung	31
9.1	Wissenschaftlicher Beitrag	31
9.2	Ausblick	31

1 Einführung

PyLab ist ein umfassendes Open-Source-Framework für Regelungstechnik, Simulation und Visualisierung in Python. Es bietet Klassen für Übertragungsfunktionen, Zustandsraummodelle, PID-Regler, Stabilitätsanalyse und MicroPython-Codegenerierung sowie eine interaktive PyQt6-Benutzeroberfläche. Im Mittelpunkt dieser Arbeit steht das in Version 2.0.0 eingeführte Verifikationsmodul: Regelkreise werden in gerichtete Signalflussgraphen überführt und mittels des Subgraph Algorithmus auf graphenstrukturelle Äquivalenz geprüft. Das Verfahren ermöglicht es, zwei Regelkreise effizient zu vergleichen – unabhängig von der konkreten Parametrierung – und arbeitet mit einer Zeitkomplexität von $O(n^3)$.

Regelungstechnische Software-Werkzeuge spielen in Forschung, Lehre und Industrie eine zentrale Rolle. Klassische Bibliotheken wie `scipy.signal` [13] oder MATLAB/Simulink [18] bieten umfangreiche numerische Verfahren, vereinen jedoch selten benutzerfreundliche grafische Oberflächen mit programmierbarer API. PyLab adressiert diesen Bedarf: Es ist als Python-Paket konzipiert, das sowohl skriptbasiert als auch interaktiv über eine PyQt6-Oberfläche genutzt werden kann.

1.1 Zielsetzung

Die Version 2.0.0 von PyLab erweitert das Framework um ein Verifikationsmodul, das zwei Regelkreise auf graphenstrukturelle Äquivalenz prüft. Die zentrale Fragestellung lautet:

Haben zwei Regelkreise – trotz verschiedener Parametrierung oder Reglerstruktur – dieselbe innere Kopplungsstruktur?

Zur Beantwortung wird der **Subgraph Algorithmus** [1] eingesetzt, der ursprünglich für den Vergleich von Abstract Syntax Trees in der Programmverifikation entwickelt wurde. PyLab überträgt dieses Konzept auf die Regelungstechnik: Die Zustandsraumdarstellung des offenen Kreises $L(s) = C(s) \cdot G(s)$ liefert einen gerichteten Graphen, dessen Adjazenzmatrix dem Algorithmus übergeben wird.

1.2 Aufbau der Arbeit

Kapitel 2 legt die regelungstechnischen und graphentheoretischen Grundlagen. Kapitel 3 beschreibt die Softwarearchitektur von PyLab. Kapitel 4 erläutert das Verifikationsverfahren im Detail. Kapitel 5 beschreibt die biologischen Äquivalenzklassen. Kapitel 6 stellt die GUI-Integration vor. Kapitel 7 analysiert Korrektheit und Komplexität. Kapitel 8 zeigt konkrete Anwendungsbeispiele. Kapitel 9 schließt mit einer Zusammenfassung.

2 Grundlagen

2.1 Regelungstechnische Grundbegriffe

Die verwendeten Begriffe orientieren sich an den Standardwerken der Regelungstechnik [2, 3, 4, 5].

Definition 1 (Übertragungsfunktion). Eine Übertragungsfunktion $G(s)$ ist der Quotient aus Ausgangs- und Eingangsgröße im Laplace-Bereich:

$$G(s) = \frac{Y(s)}{U(s)} = \frac{b_m s^m + \dots + b_1 s + b_0}{a_n s^n + \dots + a_1 s + a_0}$$

mit Zählerpolynom vom Grad m und Nennerpolynom vom Grad $n \geq m$.

Definition 2 (Offener Kreis). Der offene Kreis eines Regelkreises aus Regler $C(s)$ und Strecke $G(s)$ ist

$$L(s) = C(s) \cdot G(s).$$

Definition 3 (Zustandsraumdarstellung). Ein lineares zeitinvariantes System lässt sich in der Observable Canonical Form [2, 4] als

$$\begin{aligned}\dot{\mathbf{x}}(t) &= \mathbf{A} \mathbf{x}(t) + \mathbf{B} u(t) \\ y(t) &= \mathbf{C} \mathbf{x}(t) + D u(t)\end{aligned}$$

darstellen, wobei $\mathbf{A} \in \mathbb{R}^{n \times n}$ die *Systemmatrix* ist.

2.2 Signalflussgraph eines Regelkreises

Der Signalflussgraph wurde von Mason [9] als kompaktes Darstellungsmittel linearer Systeme eingeführt und ist seither ein etabliertes Werkzeug der Regelungstechnik [2, 3].

Definition 4 (Signalflussgraph). Der Signalflussgraph eines Regelkreises ist ein gerichteter Graph $\mathcal{G} = (V, E)$, bei dem:

- jeder Knoten $v_i \in V$ einer Zustandsvariablen x_i entspricht,
- eine Kante $(v_i, v_j) \in E$ genau dann existiert, wenn $|\mathbf{A}_{ij}| > \varepsilon$ für einen Schwellwert $\varepsilon > 0$.

Damit kodiert die binäre Adjazenzmatrix

$$\mathbf{M}_{ij} = \begin{cases} 1 & \text{falls } |\mathbf{A}_{ij}| > \varepsilon \\ 0 & \text{sonst} \end{cases}$$

die Kopplungsstruktur des offenen Kreises vollständig.

2.3 Graphentheoretische Grundbegriffe

Die verwendeten graphentheoretischen Begriffe folgen der Standardliteratur [10, 11].

Definition 5 (Adjazenzmatrix). Die Adjazenzmatrix A eines gerichteten Graphen $G = (V, E)$ mit n Knoten ist die $n \times n$ -Boole'sche Matrix mit $A_{ij} = 1$ genau dann, wenn $(v_i, v_j) \in E$.

Definition 6 (Subgraph). Ein Graph $G = (V, E)$ heißt Subgraph von $G' = (V', E')$, geschrieben $G \subseteq G'$, wenn $V \subseteq V'$ und $E \subseteq E'$.

Definition 7 (Zyklische Rotation). Eine zyklische Rotation einer Sequenz $[a_0, a_1, \dots, a_{n-1}]$ um k Positionen ergibt:

$$\text{rot}_k([a_0, \dots, a_{n-1}]) = [a_k, a_{k+1}, \dots, a_{n-1}, a_0, \dots, a_{k-1}].$$

3 Softwarearchitektur von PyLab

3.1 Modulübersicht

PyLab ist als Python-Paket `pylabb` strukturiert und gliedert sich in fünf fachliche Subpakete sowie ein GUI-Paket. Als numerische Grundlage dienen NumPy [14] und SciPy [13]; Diagramme werden mit matplotlib [15] gerendert; die GUI basiert auf PyQt6 [16]:

Subpaket	Inhalt
<code>core</code>	Übertragungsfunktion, Zustandsraum, Signale, math. Hilfsfunktionen
<code>control</code>	PID-Regler, Reglerentwurf, Stabilitätsanalyse, Regelkreis-Verifikation
<code>simulation</code>	Geschlossener Regelkreis-Simulator, Zeitbereichssimulation
<code>visualization</code>	Bode-, Nyquist-, Wurzelortskurven- und Zeitverlaufsdiagramme [15]
<code>codegen</code>	MicroPython-Codegenerator für ESP32, RP2040, STM32
<code>gui</code>	PyQt6-Hauptfenster und Widgets

Tabelle 1: Subpakete von PyLab

3.2 Paketdiagramm

Abbildung 1 zeigt das UML-Paketdiagramm von PyLab mit allen Subpaketen und ihren Abhängigkeiten.

3.3 Klassendiagramm

Das vollständige UML-Klassendiagramm (Abbildung 2) gibt einen Überblick über alle öffentlichen Klassen, ihre Attribute und Methoden sowie die Vererbungs- und Kompositionsbeziehungen.

```
../doc/uml/packages_pylabb.pdf
```

∞

../doc/uml/classes_pylabb.pdf

3.4 Kernklassen

3.4.1 TransferFunction

Die Klasse `TransferFunction` kapselt eine rationale Übertragungsfunktion durch Zähler- und Nennerkoeffizienten. Sie unterstützt algebraische Operatoren ($+$, $*$, $/$), Pol-Nullstellen-Berechnung, Bode-Werte, Impuls- und Sprungantwort sowie die Diskretisierung per Tustin- oder ZOH-Methode.

3.4.2 StateSpace

`StateSpace` repräsentiert ein System in der Form $(\mathbf{A}, \mathbf{B}, \mathbf{C}, D)$ und bietet Methoden zur Eigenwertberechnung, zur Simulation und zur Konvertierung in eine Übertragungsfunktion.

3.4.3 PIDController / DiscretePIDController

Beide Klassen implementieren kontinuierliche bzw. diskrete PID-Regelgesetze und generieren die zugehörige Übertragungsfunktion. Enthaltene Einstellregeln sind Ziegler-Nichols [7] (Sprung- und Schwingungsversuch), Cohen-Coon [8] und λ -Tuning [5].

4 Regelkreis-Verifikation

Die Verifikation von Regelkreisen ist das zentrale Thema dieser Arbeit. Das Modul `pylabb.control.verification` implementiert ein graphenstrukturelles Äquivalenzprüfungsverfahren, das auf dem Subgraph Algorithmus [1] basiert.

4.1 Grundidee

Zwei Regelkreise gelten als *graphenstrukturell äquivalent*, wenn ihre Signalflussgraphen isomorph sind oder einer den anderen als Subgraph enthält. Die Parameterwerte der Übertragungsfunktionen spielen dabei keine Rolle – es wird ausschließlich die Topologie der Zustandskopplungsstruktur verglichen.

Anwendungsszenarien:

- Prüfung, ob ein einfacherer Regelkreis (z. B. P-Regler) in einem komplexeren (z. B. PID-Regler) strukturell enthalten ist.
- Nachweis der Gleichwertigkeit zweier Entwurfsalternativen hinsichtlich ihrer inneren Dynamik.
- Automatisierte Reduktion redundanter Regelkreisentwürfe in Entwurfsdatenbanken.

4.2 Verarbeitungspipeline

Die Verifikation läuft in vier Schritten ab:

1. **Offenen Kreis berechnen:** $L_A(s) = C_A(s) \cdot G_A(s)$ und $L_B(s) = C_B(s) \cdot G_B(s)$ durch Polynommultiplikation von Zähler und Nenner.
2. **Zustandsraumdarstellung:** Scipy's `tf2ss` [13] überführt $L(s)$ in die Observable Canonical Form [2] und liefert die Systemmatrix $\mathbf{A}$.

3. **Adjazenzmatrix erzeugen:** Einträge $|A_{ij}| > \varepsilon$ werden auf 1 gesetzt, alle anderen auf 0.
4. **Subgraph Algorithmus:** Die beiden Adjazenzmatrizen werden dem Algorithmus übergeben, der die Entscheidung zurückgibt.

$$\boxed{G_A(s), C_A(s)} \rightarrow \boxed{L_A(s) = C_A \cdot G_A} \rightarrow \boxed{\mathbf{A}_A} \rightarrow \boxed{\mathbf{M}_A \in \{0, 1\}^{n \times n}} \rightarrow \text{Subgraph Algo.}$$

Abbildung 3: Verarbeitungspipeline der Regelkreis-Verifikation (für Regelkreis A)

4.3 Der Subgraph Algorithmus

Der Subgraph Algorithmus [1] prüft, ob ein Graph G in einem anderen Graphen G' enthalten ist, ohne alle $n!$ Permutationen zu enumerieren. Stattdessen werden nur n zyklische Rotationen der Adjazenzmatrix betrachtet.

4.3.1 Signaturen

Für jede Spalte j der Adjazenzmatrix wird eine eindeutige Signatur berechnet:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Lemma 1 (Eindeutigkeit der Signaturen). Die Signaturabbildung $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv.

Beweis. Der Term $\sum_i A_{ij} \cdot 2^i$ interpretiert den Spaltenvektor als Binärzahl und ist daher für unterschiedliche Muster verschieden. Der Offset $j \cdot 2^n$ verschiebt gleiche Muster an verschiedenen Spalten in disjunkte Wertebereiche $[j \cdot 2^n, (j+1) \cdot 2^n)$. $\square$

4.3.2 Zyklische Rotation

Statt alle $n!$ Knotenpermutationen zu prüfen, rotiert der Algorithmus die Spaltenreihenfolge der Adjazenzmatrix zyklisch. Dies erhält die sequentielle Ordnung des Graphen und reduziert die Suchtiefe von $O(n!)$ auf $O(n)$.

Beispiel 1. Für $n = 4$ Spalten ergeben sich exakt vier Rotationen statt $4! = 24$ Permutationen:

$$\begin{aligned} \text{Rotation 0: } & [\sigma_0, \sigma_1, \sigma_2, \sigma_3] \\ \text{Rotation 1: } & [\sigma_1, \sigma_2, \sigma_3, \sigma_0] \\ \text{Rotation 2: } & [\sigma_2, \sigma_3, \sigma_0, \sigma_1] \\ \text{Rotation 3: } & [\sigma_3, \sigma_0, \sigma_1, \sigma_2] \end{aligned}$$

4.3.3 Subgraph-Kriterium

Satz 1 (Subgraph-Kriterium). Eine Subgraph-Beziehung $G \subseteq G'$ wird genau dann erkannt, wenn die längste gemeinsame Teilsequenz (LCS) [11] der Signatur-Arrays mindestens die Länge 2 hat.

Beweis. Eine Subgraph-Beziehung erfordert mindestens eine Kante, d. h. mindestens zwei Knoten mit Verbindung. Dies entspricht einer gemeinsamen Teilsequenz der Signatur-Arrays der Länge ≥ 2 . $\square$

4.4 Entscheidungen des Algorithmus

Entscheidung	Python-Wert	Bedeutung
Äquivalent	<code>equal</code>	Beide Graphen sind strukturell identisch
Äquivalent (A bevorzugt)	<code>equal_keep_A</code>	Äquivalent; G_A hat $\geq$ so viele Kanten wie G_B
Äquivalent (B bevorzugt)	<code>equal_keep_B</code>	Äquivalent; G_B hat mehr Kanten
A enthält B	<code>"keep_A"</code>	$G_B \subseteq G_A$; Regelkreis A ist komplexer
B enthält A	<code>"keep_B"</code>	$G_A \subseteq G_B$; Regelkreis B ist komplexer
Unabhängig	<code>"keep_both"</code>	Keine strukturelle Inklusion; beide behalten

Tabelle 2: Entscheidungen des Subgraph Algorithmus in der Regelkreis-Verifikation

4.5 Implementierung: verification.py

Das Modul `pylabb/control/verification.py` stellt drei öffentliche Bezeichner bereit:

Listing 1: Öffentliche API des Verifikationsmoduls

```

1 from pylabb.control.verification import (
2     loop_to_graph,          # Regelkreis → (Adjazenzmatrix, L(s))
3     verify_loops,          # Zwei Regelkreise vergleichen
4     VerificationResult,    # Datenklasse für das Ergebnis
5 )

```

Die Funktion `loop_to_graph` wandelt einen Regelkreis in seinen Signalflussgraphen um:

Listing 2: Implementierung von `loop_to_graph`

```

1 def loop_to_graph(plant, controller, threshold=1e-9):
2     L = _open_loop(plant, controller)          # L(s) = C(s) * G(s)
3     A, _B, _C, _D = sp_sig.tf2ss(            # Observable Canonical
4         Form
5         L.num.real, L.den.real
6     )
7     adj = (np.abs(A) > threshold).astype(int)
8     return adj, L

```

Die Hauptfunktion `verify_loops` führt die vollständige Verifikation durch:

Listing 3: Implementierung von `verify_loops` (gekürzt)

```
1 def verify_loops(plant_A, controller_A, plant_B, controller_B,
2   ...):
3     from subgraph import Subgraph # ImportError wenn nicht
4     installiert
5
6     adj_A, L_A = loop_to_graph(plant_A, controller_A)
7     adj_B, L_B = loop_to_graph(plant_B, controller_B)
8
9     # Matrizen auf gemeinsame Größe bringen (Zero-Padding)
10    if adj_A.shape[0] != adj_B.shape[0]:
11        n_max = max(adj_A.shape[0], adj_B.shape[0])
12        # ... zero-padding auf n_max x n_max ...
13
14    algo = Subgraph()
15    decision, kept = algo.compare_graphs(mat_A.astype(float),
16                                         mat_B.astype(float))
17    return VerificationResult(decision=decision, ...)
```

4.6 Datenklasse `VerificationResult`

Das Ergebnisobjekt kapselt alle relevanten Informationen:

Listing 4: `VerificationResult`-Klasse

```
1 @dataclass
2 class VerificationResult:
3     decision: str # Algorithmus-Entscheidung
4     matrix_A: NDArray[np.int_] # Adjazenzmatrix
5     Regelkreis A
6     matrix_B: NDArray[np.int_] # Adjazenzmatrix
7     Regelkreis B
8     kept_matrix: Optional[NDArray] # Bevorzugte Matrix (oder
9     None)
10    open_loop_A: TransferFunction # L_A(s)
11    open_loop_B: TransferFunction # L_B(s)
12    label_A: str # Beschriftung A
13    label_B: str # Beschriftung B
14
15    # Bequemlichkeits-Properties
16    @property
17    def are_equivalent(self) → bool: ...
18    @property
19    def a_contains_b(self) → bool: ...
20    @property
21    def b_contains_a(self) → bool: ...
22    @property
23    def are_independent(self) → bool: ...
24    @property
25    def summary(self) → str: ...
```

4.7 Padding unterschiedlich großer Graphen

Haben die beiden offenen Kreise verschiedene Ordnungen $n_A \neq n_B$, werden beide Adjazenzmatrizen auf die Größe $n_{\max} \times n_{\max}$ mit Nullen aufgefüllt:

$$\mathbf{M}_A^{\text{pad}} = \begin{pmatrix} \mathbf{M}_A & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{pmatrix} \in \{0, 1\}^{n_{\max} \times n_{\max}}$$

Dies stellt sicher, dass der Subgraph Algorithmus stets auf gleich großen Matrizen operiert, während die ursprüngliche Knotenanzahl im Ergebnisobjekt erhalten bleibt.

5 Biologisch orientierte Äquivalenzklassen von Regelkreisen

5.1 Motivation und Erkenntnisziel

Die Regelungstechnik ist eine abstrakte Disziplin: Übertragungsfunktionen, Zustandsraumdarstellungen und Signalflussgraphen beschreiben das dynamische Verhalten technischer Systeme in einer Sprache der Mathematik. Gleichzeitig hat die Natur im Laufe von Millionen Jahren der Evolution hocheffiziente Regelkreise entwickelt – in Tieren, Organen und Zellen – die technischen Systemen in ihrer Leistungsfähigkeit, Robustheit und Adaptivität weit überlegen sind.

Das Ziel dieses Kapitels ist es, eine systematische Klassifikation von technischen Regelkreisen *nach biologischen Vorbildern* zu entwickeln. Dabei wird folgende Kernthese verfolgt:

Regelkreise, die graphenstrukturell äquivalent sind (im Sinne des Subgraph Algorithmus aus Kapitel 4), können einer gemeinsamen biologischen Regelklasse zugeordnet werden. Diese Zuordnung eröffnet die Möglichkeit, aus dem Verhalten des biologischen Vorbildes erweiterte Funktionen und Entwurfsstrategien für den technischen Regelkreis abzuleiten.

Die Idee verbindet zwei Forschungsrichtungen:

- die **Bionik der Regelungstechnik**, also die Übertragung biologischer Prinzipien auf technische Systeme [2, 3], und
- die **graphenstrukturelle Verifikation** aus Kapitel 4, die eine formale Äquivalenzprüfung von Regelkreisen ermöglicht.

Warum biologische Vorbilder? Lebewesen sind evolutionär optimierte Regelungssysteme. Das menschliche Auge kompensiert Beleuchtungsänderungen über sechs Dekaden, die Körpertemperatur wird auf $\pm 0,5^\circ\text{C}$ gehalten, das vestibuläre System balanciert einen aufrecht gehenden Menschen gegen Störungen – alles ohne explizit programmierten Reglerentwurf. Diese biologischen Systeme verkörpern *optimale Lösungen* regelungstechnischer Probleme. Wer versteht, welcher abstrakte Regelkreistyp einem biologischen Organ entspricht, kann aus dem biologischen Verhalten qualitative und quantitative Entwurfshinweise für technische Systeme gewinnen.

Abgrenzung. Dieses Kapitel ist *keine* Einführung in die Neurobiologie oder Physiologie. Es betrachtet biologische Systeme ausschließlich aus der Perspektive der Regelungstechnik: als Regelstrecken, Regler, Sollwertgeber und Rückführzweige. Die biologischen Beschreibungen sind vereinfacht und dienen dem Zweck der regelungstechnischen Analogiebildung.

5.2 Grundlegende Konzepte der biologischen Regelung

5.2.1 Hierarchische Regelkreisstruktur in Lebewesen

Biologische Regelungssysteme sind grundsätzlich hierarchisch organisiert. In einem Säugetier lassen sich mindestens vier Regelungsebenen unterscheiden:

Ebene	Biologisches System	Regelungstechnisches Äquivalent
4	Kortikale Verhaltenssteuerung	Hierarchischer Mehrgrößenregler (MIMO), Sollwertvorgabe
3	Hormonelles System	Langsamer integrierender Regler mit großer Totzeit
2	Autonomes Nervensystem	Schneller nichtlinearer Rückführregler
1	Zelluläre Homöostase	Lokaler Proportionalregler mit Sättigung

Tabelle 3: Hierarchische Regelungsebenen in Säugetieren

Jede dieser Ebenen weist charakteristische dynamische Eigenschaften auf: unterschiedliche Zeitkonstanten (Millisekunden bis Tage), unterschiedliche Reglerstrukturen (P, PI, PID, nichtlinear) und unterschiedliche Signaltypen (elektrische Impulse, chemische Gradienten, mechanische Kraft).

5.2.2 Gemeinsame Eigenschaften biologischer Regelkreise

Biologische Regelkreise teilen eine Reihe von Eigenschaften, die für die Klassifikation relevant sind:

Robustheit gegenüber Parameterunsicherheit. Biologische Systeme funktionieren trotz erheblicher Variabilität in Parametern (Körpermasse, Körpertemperatur, Alter). Dies entspricht dem Entwurfsziel *robuste Regelung* in der Regelungstechnik.

Adaptivität. Biologische Regler passen ihre Parameter an veränderte Bedingungen an (Lernen, Akklimatisation). Dies entspricht *adaptiver Regelung* oder *selbsteinstellenden Reglern*.

Vorsteuersignal (Feedforward). Viele biologische Systeme reagieren nicht nur rückgekoppelt, sondern antizipieren Störungen. Das Kleinhirn beispielsweise berechnet motorische Korrekturen *vor* der Ausführung einer Bewegung. Dies entspricht einer *Vorsteuerung* oder *Smith-Prädiktor*-Struktur in der technischen Regelungstechnik.

Mehrfachrückkopplung. Biologische Regelkreise besitzen fast immer mehrere Rückkopplungspfade auf verschiedenen Zeitskalen. Dies entspricht *kaskadiert verschalteten Regelkreisen*.

Nichtlinearität. Sättigung, Schwellwerte (z. B. Aktionspotential) und Hysterese sind biologischen Systemen inhärent. Entsprechend enthält jede biologisch motivierte Regelklasse nichtlineare Erweiterungen des Grundtyps.

5.3 Definition der biologischen Äquivalenzklassen

Definition 8 (Biologische Äquivalenzklasse). Zwei Regelkreise $\mathcal{L}_A = (C_A(s), G_A(s))$ und $\mathcal{L}_B = (C_B(s), G_B(s))$ gehören zur selben *biologischen Äquivalenzklasse* $\mathcal{K}_\beta$, wenn:

1. ihre Signalflussgraphen graphenstrukturell äquivalent sind (gemäß Subgraph Algorithmus, Kapitel 4),
2. die charakteristischen Frequenzeigenschaften (Bandbreite, Phasenlage, Integralanteil) mit dem biologischen Vorbild β übereinstimmen, und
3. die qualitativen Stellgrößen- und Regelgrößenverläufe dem biologischen Regelverhalten entsprechen.

Das biologische Vorbild β ist das Tier, Organ oder Teilsystem, dessen Regelstruktur als Referenz dient.

Im Folgenden werden sieben biologische Äquivalenzklassen ausführlich beschrieben. Jede Klasse ist nach einem biologischen Vorbild benannt, das die wesentlichen Eigenschaften des zugehörigen Regelkreistyps exemplarisch verkörpert.

5.4 Klasse I – Der Seestern: Dezentrale Proportionalregelung

5.4.1 Biologisches Vorbild: Seestern (*Asteroidea*)

Der Seestern ist ein Stachelhäuter ohne zentrales Gehirn. Seine Fortbewegung wird durch ein dezentrales Nervennetz gesteuert: Jeder der fünf Arme enthält ein eigenes Ganglion, das die Tube-Feet (Saugnapffüße) des betreffenden Arms autonom koordiniert. Eine zentrale Instanz, die alle Arme zusammenführt, existiert nicht – trotzdem bewegt sich der Seestern gerichtet und koordiniert.

Regelungstechnische Interpretation. Jeder Arm des Seesterns realisiert einen *lokalen Proportionalregler*: Der Stellreflex auf Berührung oder optische Gradienten erfolgt unmittelbar, ohne zeitliche Integration. Die Koordination ergibt sich aus dem Kopplungsterm zwischen den Armen: Wenn Arm 1 eine Bewegungsrichtung “wählt”, werden die übrigen Arme durch mechanische und neuronale Kopplung in dieselbe Richtung gezogen.

Biologische Regelklasse 1 (Seestern – Dezentrale Proportionalregelung). Die Seestern-Klasse umfasst alle Regelkreise mit folgenden Merkmalen:

- Rein proportionaler Regelanteil: Übertragungsfunktion des Reglers $C(s) = K_P$ (reelle Konstante).
- Kein Integralterm, kein bleibender Ausgleich.
- Signalflussgraph: Kettenstruktur oder vollständig verbundener Multigraph mit ausschließlich direkten Kanten (keine Rückkopplungsschleifen zweiter Ordnung).
- Adjazenzmatrix: dünn besetzt (wenige Kanten), reguläre Struktur (jeder Knoten hat denselben Eingrad und Ausgrad).

5.4.2 Graphenstruktur der Seestern-Klasse

Die Systemmatrix $\mathbf{A}$ eines P-Reglers mit einer Strecke n -ter Ordnung in Observable Canonical Form hat folgende Struktur:

$$\mathbf{A}_{\text{Seestern}} = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 1 \\ -a_0 & -a_1 & \cdots & -a_{n-2} & -a_{n-1} \end{pmatrix}$$

Diese Matrix erzeugt einen *gerichteten Pfadgraphen* P_n mit einer einzigen Rückführkante von v_n nach $v_1, \dots, v_n$. Die Adjazenzmatrix $\mathbf{M}_{\text{Seestern}}$ ist eine obere Bidiagonale mit einer voll besetzten letzten Zeile – das charakteristische “Seestern-Muster”.

5.4.3 Abgeleitete Entwurfsstrategien

Die biologische Analogie offenbart folgende Entwurfshinweise:

1. **Verteilte Implementierung.** Wie beim Seestern können P-Regler für verteilte Systeme (z. B. mehrere ESP32-Knoten in einem Sensornetz) ohne zentrale Koordination implementiert werden. Jeder Knoten regelt seine lokale Größe; die Kopplung erfolgt durch gemeinsame physikalische Größen (Spannung, Druck) – nicht durch explizite Kommunikation.
2. **Fehlertoleranz.** Der Seestern überlebt den Verlust eines Arms. Entsprechend sollten P-Regler in sicherheitskritischen Systemen redundant ausgelegt werden: Fällt ein Regler-Knoten aus, übernehmen die verbleibenden die Regelung mit reduzierter Stellkraft.
3. **Robuste Inbetriebnahme.** Ein P-Regler mit kleinem K_P ist immer stabil (für stabile Strecken). Wie der Seestern “ausprobiert”, in welche Richtung er sich bewegen soll, kann K_P iterativ erhöht werden, bis die gewünschte Regeldynamik erreicht ist.

5.4.4 Erweiterungspotenzial aus der biologischen Analogie

Die dezentrale Intelligenz des Seesterns legt nahe:

Hypothese 1 (Seestern-Erweiterung). Ein Regelkreis der Seestern-Klasse kann durch Hinzufügen von *Kommunikationspfaden* zwischen bisher isolierten P-Reglern zu einem *konsens-basierten Mehrfachregler* erweitert werden. Die erweiterte Funktion ist die kollektive Optimierung eines gemeinsamen Gütekriterials ohne zentrale Instanz – analog zu Ant-Colony-Optimierungsverfahren.

5.5 Klasse II – Pupille: Integrierender Regelkreis mit Sättigung

5.5.1 Biologisches Vorbild: Pupillenreflex des Auges

Der Pupillenreflex ist einer der am besten verstandenen Regelkreise in der Biologie [5]. Er regelt den Lichteinfall auf die Netzhaut durch Verstellung des Pupillendurchmessers. Die Regelgröße ist die Beleuchtungsstärke auf der Netzhaut, die Stellgröße ist die Kontraktion des Musculus sphincter pupillae (Verengung) oder des Musculus dilatator pupillae (Erweiterung).

Dynamisches Verhalten. Das Pupillensystem zeigt folgende charakteristische Merkmale:

- **Totzeit:** ca. 0,18 s (Latenz der Photorezeptoren und Nervenleitung).
- **Zeitkonstante:** ca. 0,1 s (mechanische Reaktion des Muskels).
- **Sättigung:** Pupillendurchmesser begrenzt zwischen 1,5 mm (maximale Kontraktion) und 8 mm (maximale Dilatation) – ein Bereich von ca. 28 dB Dämpfung.
- **Nichtlinearität:** Die Helligkeitswahrnehmung folgt dem Weber-Fechner-Gesetz (logarithmisch), was die effektive Verstärkung bei hohen Lichtintensitäten reduziert.

Biologische Regelklasse 2 (Pupille – PI-Regelung mit Totzeit und Sättigung). Die Pupillen-Klasse umfasst alle Regelkreise mit:

- PI-Regler: $C(s) = K_P \left(1 + \frac{1}{T_I s}\right)$, wobei der Integralterm den bleibenden Regelungsfehler eliminiert.
- Totzeitglied in der Strecke: $G(s) = G_0(s) \cdot e^{-T_t s}$ mit $T_t > 0$.
- Stellgrößenbeschränkung (Sättigung): $|u(t)| \leq u_{\max}$.
- Signalflussgraph: Kettenstruktur mit zusätzlichem selbstrückführendem Knoten (der Integralzustand erzeugt eine Schleife $v_1 \rightarrow v_1$).

5.5.2 Graphenstruktur der Pupillen-Klasse

Der Integralanteil eines PI-Reglers fügt einen Zustandsknoten v_I hinzu, der ausschließlich Selbstkopplung hat (Integralzustand $\dot{x}_I = e$, $x_I \rightarrow x_I$). Die Adjazenzmatrix enthält daher einen Diagonaleintrag:

$$\mathbf{M}_{\text{Pupille}} = \begin{pmatrix} 1 & 0 & 0 & \cdots \\ 1 & 0 & 1 & \cdots \\ 0 & 0 & 0 & \ddots \\ \vdots & \vdots & \ddots & \ddots \end{pmatrix}$$

Die Selbstschleife bei v_I (Eintrag $M_{11} = 1$) ist das graphische Erkennungsmerkmal der Pupillen-Klasse und unterscheidet sie von der Seestern-Klasse durch genau einen zusätzlichen Diagonaleintrag.

5.5.3 Abgeleitete Entwurfsstrategien

1. **Anti-Windup als biologisches Prinzip.** Das Auge verhindert “Integratorsättigung” durch neuronale Anpassung: Bei maximaler Pupillenverengung wird das Integrations-signal nicht weiter aufgestapelt. Für technische PI-Regler in PyLab sollte daher stets ein Anti-Windup-Mechanismus implementiert werden, sobald eine Sättigungsgrenze erkennbar ist.
2. **Totzeitkompensation nach Smith.** Die biologische Totzeit des Pupillenreflexes führt zu Oszillationen bei schnellen Lichtsprüngen (Hippus-Phänomen). Technische Systeme der Pupillen-Klasse mit signifikanter Totzeit sollten mit einem Smith-Prädiktor erweitert werden, der die Totzeit aus der Rückkopplungsschleife “herausnimmt”.

3. **Logarithmische Skalierung der Eingangsgröße.** Das Weber-Fechner-Gesetz legt nahe, dass Regelstrecken mit sehr großem Dynamikbereich (mehr als 40 dB) von einer logarithmischen Vorverarbeitung profitieren, da dies die effektive Streckenverstärkung über den gesamten Arbeitsbereich konstant hält.

5.5.4 Erweiterungspotenzial: Adaptiver PI-Regler

Hypothese 2 (Pupillen-Erweiterung). Die logarithmische Nichtlinearität des Pupillenreflexes liefert die Entwurfsregel: Für Regelstrecken mit mehr als 40 dB Dynamikbereich soll die Reglerverstärkung K_P als Funktion des aktuellen Betriebspunktes adaptiert werden:

$$K_P(y) = \frac{K_{P,0}}{1 + \alpha \ln(1 + y/y_0)}$$

wobei α den Grad der Nichtlinearität und y_0 den Nominalwert der Regelgröße beschreibt. Dies entspricht einer *gainscheduled PI-Regelung*, motiviert durch das biologische Vorbild.

5.6 Klasse III – Das Kleinhirn: Vorsteuerung und Lernender Regler

5.6.1 Biologisches Vorbild: Kleinhirn (*Cerebellum*)

Das Kleinhirn ist das Koordinationszentrum für motorische Funktionen beim Menschen und bei Wirbeltieren. Es enthält etwa 50–80 % aller Neuronen des menschlichen Gehirns und ist für die Feinabstimmung von Bewegungen verantwortlich. Das Kleinhirn arbeitet als *internes Modell* des Bewegungsapparates: Es lernt im Laufe der Entwicklung, Korrekturbefehle *vor* der eigentlichen Bewegungsausführung zu berechnen.

Regelungstechnische Charakteristika.

- **Vorsteuerung (Feedforward):** Das Kleinhirn berechnet die erforderliche Muskelspannung *auf Basis des Sollwertes*, nicht auf Basis des gemessenen Fehlers. Dies ist eine klassische Vorsteuerstruktur.
- **Lernmechanismus:** Durch Purkinje-Zellen und Kletterfasern wird das interne Modell iterativ verbessert – ein biologisches Äquivalent zu iterativen Lernregelungen (*Iterative Learning Control, ILC*).
- **Zeitkonstanten:** Motorische Korrekturen geschehen in 50–100 ms; die Lernphase (Motorisches Gedächtnis) über Wochen bis Jahre.
- **Störungsbeobachter:** Das Kleinhirn schätzt externe Störungen (Gewicht eines Objektes, Wind) anhand von Erwartungsmodellen.

Biologische Regelklasse 3 (Kleinhirn – PID mit Vorsteuerung und iterativem Lernen). Die Kleinhirn-Klasse umfasst alle Regelkreise mit:

- PID-Rückführregler: $C(s) = K_P + \frac{K_I}{s} + K_D s$.
- Additiver Vorsteuerung $U_{FF}(s) = F(s) \cdot W(s)$, wobei $F(s)$ das Vorsteuermodell und $W(s)$ den Sollwert bezeichnet.

- Einer Lernkomponente $\Delta U_k = L \cdot e_{k-1}$ (iterative Korrektur über Wiederholungszyklen k).
- Signalflussgraph: Komplexer Graph mit parallelen Pfaden (Rückkopplung und Vorwärtspfad) sowie einem separaten Speicherknoten für den Lernterm.

5.6.2 Graphenstruktur der Kleinhirn-Klasse

Die Kleinhirn-Klasse zeichnet sich durch einen Signalflussgraphen mit *zwei parallelen Hauptpfaden* aus: einem Rückkopplungspfad (PID) und einem Vorwärtspfad (Vorsteuerung). Die Adjazenzmatrix hat dementsprechend Block-Struktur:

$$\mathbf{M}_{\text{Kleinhirn}} = \begin{pmatrix} \mathbf{M}_{\text{PID}} & \mathbf{0} \\ \mathbf{C}_{\text{FF}} & \mathbf{M}_{\text{FF}} \end{pmatrix}$$

wobei $\mathbf{M}_{\text{PID}}$ die Adjazenzmatrix des PID-Kreises, $\mathbf{M}_{\text{FF}}$ die des Vorsteuermodells und $\mathbf{C}_{\text{FF}}$ die Kopplung zwischen beiden darstellt. Der Subgraph Algorithmus erkennt $\mathbf{M}_{\text{PID}}$ als Subgraph von $\mathbf{M}_{\text{Kleinhirn}}$: Ein PID-Kreis ist in jedem Kleinhirn-Kreis enthalten.

5.6.3 Abgeleitete Entwurfsstrategien

1. **Iterative Learning Control (ILC).** Das Kleinhirn lernt aus jedem Bewegungszyklus. In technischen Anwendungen mit *repetitiven Prozessen* (Roboterschweißen, Druckmaschinen, Windturbinen) sollte jeder Zyklus k dazu genutzt werden, den Vorsteuerterm zu verbessern:

$$u_{k+1}(t) = u_k(t) + \gamma \cdot e_k(t)$$

mit Lernrate $\gamma \in (0, 1)$. PyLab kann einen solchen Lernterm als separaten Signalpfad im Signalflussgraphen abbilden.

2. **Modellbasierte Vorsteuerung.** Ist ein Streckenmodell $\hat{G}(s)$ bekannt, lautet das optimale Vorsteuerglied $F(s) = \hat{G}^{-1}(s)$ (Streckeninverse). In der Praxis wird $F(s)$ auf einen kausalen, properen Anteil beschränkt. PyLab kann prüfen, ob die Vorsteuerstruktur zur Kleinhirn-Klasse gehört, indem es den erweiterten Graphen mit der Referenztopologie vergleicht.
3. **Schätzung von Störmomenten.** Das Kleinhirn schätzt Trägheitskräfte, die bei der Bewegung auftreten. Analog sollte in robotischen Anwendungen ein *Störgrößenbeobachter* (Disturbance Observer) dem PID-Regler überlagert werden.

5.6.4 Erweiterungspotenzial: Biologisch motiviertes ILC-Modul

Hypothese 3 (Kleinhirn-Erweiterung). Für Regelkreise der Kleinhirn-Klasse, die in PyLab identifiziert werden, soll automatisch ein ILC-Modul (`pylabb.control.ilc`) generiert werden, das:

- den Vorsteuerterm $u_{\text{FF},k}$ über Wiederholungszyklen lernt,
- die Konvergenz des Lernfehlers $\|e_k\|_2 \rightarrow 0$ überprüft, und
- MicroPython-Code für die Lernkomponente auf einem ESP32 generiert.

Dies ist eine direkte Erweiterung des `genmicropy`-Codegenerators (Kapitel 3) um biologisch motivierte Lernalgorithmen.

5.7 Klasse IV – Das Herz: Autonomer Taktgeber und Kaskadenregelung

5.7.1 Biologisches Vorbild: Herzkreislaufsystem

Das menschliche Herz-Kreislauf-System ist ein Musterbeispiel für *Kaskadenregelung* mit autonomem Taktgeber. Der Sinusknoten generiert ohne äußere Einwirkung Aktionspotenziale mit einer Grundfrequenz von $60\text{--}100\text{ min}^{-1}$. Das autonome Nervensystem (Sympathikus/Parasympathikus) überlagert diesen Taktgeber mit einem Korrektursignal, das die Herzrate an den Bedarf (körperliche Aktivität, Stress) anpasst.

Kaskadenstruktur.

- **Äußerer Regelkreis (langsam):** Sauerstoffversorgung des Gewebes als Regelgröße; Regelgröße ist der mittlere arterielle Druck (MAP). Zeitkonstante: 20–60 s.
- **Innerer Regelkreis (schnell):** Herzrate als Stellgröße für den Druck; Herzrate wird durch den Sinusknoten direkt eingestellt. Zeitkonstante: 1–5 s.
- **Barorezeptor-Reflex:** Drucksensoren in der Aortenbogenregion messen den Blutdruck direkt und schicken ein Korrektursignal an den Sinusknoten – ein hochdynamischer lokaler Regelkreis mit Zeitkonstante $< 1\text{ s}$.

Biologische Regelklasse 4 (Herz – Kaskadenregelung mit autonomem Taktgeber). Die Herz-Klasse umfasst alle Regelkreise mit:

- Kaskadenstruktur: Ein äußerer PI-Regler $C_{\text{aussen}}(s)$ gibt den Sollwert für einen inneren P-Regler $C_{\text{innen}}(s)$ vor.
- Innerer Regelkreis mit deutlich größerer Bandbreite als äußerer. Als Faustregel gilt: $\omega_{\text{innen}} \geq 5 \omega_{\text{aussen}}$.
- Einem autonomen Taktgeber (Oszillatorterm) im Signalflussgraphen.
- Signalflussgraph: Zwei verschachtelte Schleifen (innere und äußere Rückkopplung) mit einem zusätzlichen Oszillatorknoten.

5.7.2 Graphenstruktur der Herz-Klasse

Die Kaskadenstruktur erzeugt einen Signalflussgraphen mit *zwei konzentrischen Schleifen*. Die Adjazenzmatrix hat Block-Tridiagonalstruktur:

$$\mathbf{M}_{\text{Herz}} = \begin{pmatrix} \mathbf{M}_{\text{innen}} & \mathbf{K}_{ai} & \mathbf{0} \\ \mathbf{0} & \mathbf{M}_{\text{aussen}} & \mathbf{0} \\ \mathbf{K}_{oi} & \mathbf{0} & \mathbf{M}_{\text{osc}} \end{pmatrix}$$

wobei $\mathbf{K}_{ai}$ die Kopplung vom äußeren zum inneren Regler, $\mathbf{K}_{oi}$ die Kopplung vom Oszillator zum inneren Kreis und $\mathbf{M}_{\text{osc}} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$ die Adjazenzmatrix eines harmonischen Oszillators beschreibt.

5.7.3 Abgeleitete Entwurfsstrategien

1. **Kaskadenregelung als Standardstruktur.** Der Herzkreislauf bestätigt das Entwurfsprinzip der Kaskadenregelung: Schnelle innere Störungen (Herzrhythmus) werden durch den inneren Kreis bereits kompensiert, bevor sie den äußeren Kreis beeinflussen. In technischen Systemen (z. B. Antriebsregelung: innerer Stromregelkreis, äußerer Drehzahlregelkreis) sollte dieselbe Hierarchie eingehalten werden.
2. **Entwurf des inneren Kreises zuerst.** Das Herz entwirft den Sinusknoten evolutionär früher als die übergeordneten Regulierungsmechanismen. Analog ist in der technischen Kaskadenregelung stets der innere Regelkreis zuerst einzustellen (Ziegler-Nichols oder IMC [5]), bevor der äußere Regler parametrisiert wird.
3. **Überwachung des Takt-Verhältnisses.** Der Sinusknoten wird überwacht (EKG). Analog sollte PyLab bei Kaskadenregelkreisen die Verhältnisse der Bandbreiten automatisch prüfen und eine Warnung ausgeben, wenn $\omega_{\text{innen}} < 3 \omega_{\text{ausen}}$ (drohende Instabilität des Kaskadensystems).

5.7.4 Erweiterungspotenzial: Pulsweitenmodulierter Antrieb

Hypothese 4 (Herz-Erweiterung). Das Herz moduliert die Herzrate (Frequenz) statt der Kontraktionsstärke (Amplitude). Dies entspricht Pulsweitenmodulation (PWM). Kaskadenregelkreise der Herz-Klasse, die auf Mikrocontrollern implementiert werden, sollten daher:

- den inneren Regelkreis als PWM-Regler mit variabler Frequenz implementieren,
- die Ausgangsperiode dynamisch an die Regelabweichung anpassen,
- und den äußeren PI-Regler mit deutlich kleinerer Abtastrate (mindestens Faktor 10) betreiben.

genmipy soll diese Struktur als biologisch motivierten “Herz-Template” automatisch generieren.

5.8 Klasse V – Der Albatros: Energieoptimale Regelung mit Windausnutzung

5.8.1 Biologisches Vorbild: Albatros (*Diomedea*)

Der Albatros ist der Meister der energieeffizienten Fortbewegung. Er verbringt den größten Teil seines Lebens auf See und durchfliegt dabei Zehntausende von Kilometern – mit einem Energieverbrauch, der deutlich unter dem aller anderen Vögel gleicher Größe liegt. Sein Geheimnis ist das *dynamische Segeln*: Der Albatros nutzt den Windgradienten über der Meeresoberfläche (schneller Wind oben, langsamerer Wind in Oberflächennähe) aus, indem er kontrolliert zwischen diesen Schichten wechselt und dabei kinetische Energie aus dem Wind “erntet”.

Regelungstechnische Interpretation. Das dynamische Segeln ist eine *zeitoptimale energieerntende Steuerung*. Der Albatros maximiert $\int_{t_0}^{t_1} \mathbf{v}_{\text{wind}} \cdot \mathbf{v}_{\text{Albatros}} dt$ unter der Nebenbedingung einer Mindestflughöhe. Dies entspricht einem *optimalen Regler* (LQR oder Pontryaginsches Maximumprinzip) mit einer Nebenbedingung (Constraint).

Biologische Regelklasse 5 (Albatros – LQR mit Nebenbedingungen und Störgrößenaufschaltung). Die Albatros-Klasse umfasst alle Regelkreise mit:

- Zustandsrückführung durch einen optimal ausgelegten Rückführvektor: $C(s) \equiv \mathbf{k}^T$ (LQR-Regler).
- Störgrößenaufschaltung: Die messbare Störgröße $d(t)$ (Windgeschwindigkeit, externe Kraft) wird direkt in die Stellgröße eingerechnet.
- Beschränkter Stellgröße oder Zustand ($u_{\min} \leq u \leq u_{\max}$, $x_{\min} \leq x \leq x_{\max}$).
- Signalflussgraph: Vollständig verbundener Graph mit zusätzlichem Störeingangsknoten (Pendant zu den “Wind”-Knoten des Albatros).

5.8.2 Graphenstruktur der Albatros-Klasse

Ein LQR-Regler implementiert eine Zustandsrückkopplung $u = -\mathbf{k}^T \mathbf{x}$, was im geschlossenen Kreis zu einer vollständig verbundenen letzten Zeile der Systemmatrix führt. Die Adjazenzmatrix der Albatros-Klasse hat daher *mindestens eine vollständig besetzte Zeile* (die Rückführzeile):

$$\mathbf{M}_{\text{Albatros}} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \quad (\text{Beispiel, } n = 4)$$

Hinzu kommt eine Spalte für den Störgrößenknoten v_d , der mit allen Zustandsknoten verbunden ist (Albatros “sieht” den Wind an jedem Ort).

5.8.3 Abgeleitete Entwurfsstrategien

1. **Messbare Störgrößen nutzen.** Der Albatros misst den Wind explizit (über Druckrezeptoren am Schnabel und den Stutzknochen der Flügelwurzel). In technischen Systemen sollten messbare Störgrößen (Netzfrequenz, Umgebungstemperatur, externe Kraft) stets direkt in den Regler aufgenommen werden – Störgrößenaufschaltung reduziert die erforderliche Regelverstärkung erheblich.
2. **Energieoptimale Trajektorienplanung.** Das Segment-Segelerhalten des Albatros entspricht einer *Model Predictive Control* (MPC): In jedem Zeitschritt wird über einen Horizont H optimiert, welcher Flugpfad die maximale Energieausbeute liefert. PyLab könnte dies als MPC-Modul implementieren.
3. **Ausbeutung periodischer Störungen.** Meereswellen (und damit der Windgradient) sind näherungsweise periodisch. Ein Resonanzfilter im Vorwärtspfad, der auf die dominante Wellenfrequenz abgestimmt ist, kann die Energieernte weiter steigern – analoges biologisches Verhalten ist beim Albatros als “Resonanzsegeln” bekannt.

5.8.4 Erweiterungspotenzial: MPC-Modul für PyLab

Hypothese 5 (Albatros-Erweiterung). Regelkreise der Albatros-Klasse werden in PyLab automatisch als Kandidaten für eine MPC-Erweiterung identifiziert. Die Identifikation erfolgt durch:

1. Nachweis, dass die Adjazenzmatrix einen Störeingangsknoten enthält (Subgraph-Prüfung gegen eine Referenztopologie).
2. Nachweis, dass die Rückführzeile vollständig besetzt ist (LQR-Struktur).

Sind beide Bedingungen erfüllt, schlägt PyLab eine MPC-Parametrierung vor und berechnet den LQR-Verstärkungsvektor $\mathbf{k}$ über die Riccati-Gleichung.

5.9 Klasse VI – Die Mimose: Nichtlinearer Regelkreis mit Hysterese

5.9.1 Biologisches Vorbild: Mimose (*Mimosa pudica*)

Die Mimose ist eine Pflanze, die auf Berührungsreize mit dem blitzschnellen Zusammenfallen ihrer Blätter reagiert (Seismonastie). Dieses Verhalten ist eines der bekanntesten Beispiele für pflanzliche Reiz-Reaktions-Systeme. Aus regelungstechnischer Sicht zeigt die Mimose folgende charakteristische Eigenschaften:

Schwellwertverhalten (Trigger). Die Mimose reagiert erst ab einem *Mindestierreiz* (Schwellwert). Kleine Berührungen werden ignoriert – ein klares Schwellwertverhalten, das dem Schaltelement (Zweipunktregler, Schmitt-Trigger) entspricht.

Hysterese. Nach dem Zusammenfallen öffnen sich die Blätter erst nach einer Erholungszeit von mehreren Minuten wieder. Dies ist eine ausgeprägte Hysterese: Der Schließzustand wird erst verlassen, wenn eine deutlich längere Beruhigungszeit vergangen ist als die initiale Reizzeit.

Ermüdung (Refractory Period). Wird die Mimose wiederholt stimuliert, wird die Reaktion schwächer und schließlich bleibt sie aus – eine Adaptation (Habituation), die dem Anti-Windup-Mechanismus technischer Zweipunktregler ähnelt.

Biologische Regelklasse 6 (Mimose – Nichtlinearer Zweipunktregler mit Hysterese). Die Mimosen-Klasse umfasst alle Regelkreise mit:

- Zweipunktregler (Bang-Bang-Regler) mit Hysterese-Bandbreite h :

$$u(t) = \begin{cases} +U_{\max} & \text{wenn } e(t) > +h/2 \\ -U_{\max} & \text{wenn } e(t) < -h/2 \\ u(t^-) & \text{sonst (Hysterese)} \end{cases}$$

- Einem Ermüdungsterm: Nach N Schaltzyklen wird $U_{\max}$ auf $U_{\max} \cdot \eta^N$ reduziert ($\eta < 1$).
- Signalflussgraph: Stark nichtlinearer Signalflussgraph mit einem Schaltknoten (diskontinuierliche Übertragungsfunktion). Die Adjazenzmatrix ist äquivalent zur Seestern-Klasse, jedoch mit einem markierten Schaltknoten.

5.9.2 Graphenstruktur der Mimosen-Klasse

Der Zweipunktregler selbst hat keine rationale Übertragungsfunktion und lässt sich daher nicht direkt über eine Zustandsraumdarstellung erfassen. Stattdessen wird die *lineare Näherung* (Äquivalente Linearisierung via Describing Function) verwendet:

$$C_{\text{eq}}(j\omega, a) = \frac{4U_{\text{max}}}{\pi a} \sqrt{1 - \left(\frac{h}{a}\right)^2} - j \frac{4U_{\text{max}}h}{\pi a^2}$$

Diese komplexwertige Verstärkung erzeugt eine Adjazenzmatrix, die strukturell der Seestern-Klasse entspricht – aber durch einen imaginären Anteil (Phasenverschiebung durch Hysterese) ergänzt wird. PyLab identifiziert Mimosen-Regelkreise durch:

1. Strukturelle Äquivalenz zur Seestern-Klasse (P-Regler-Topologie), und
2. Nachweis eines Hysterese-Parameters $h > 0$ in der Parametrierung.

5.9.3 Abgeleitete Entwurfsstrategien

1. **Optimale Hysteresebandbreite.** Die Mimose wählt ihre Hysteresebandbreite evolutionär optimal: klein genug, um auf echte Bedrohungen zu reagieren, groß genug, um Windböen zu ignorieren. Für technische Zweipunktregler (Thermostate, Druckregler) lautet die optimale Hysterese-Regel: $h \approx 2\sigma_n$, wobei σ_n die Standardabweichung des Messrauschens ist.
2. **Grenzzyklen als Designmerkmal.** Ein Zweipunktregler mit symmetrischer Strecke erzeugt immer einen Grenzzyklus (Daueroszillation). Die Amplitude und Frequenz des Grenzzyklus sind durch die Describing Function berechenbar. PyLab soll für Mimosen-Klassen-Regelkreise automatisch den resultierenden Grenzzyklus berechnen und anzeigen.
3. **Habituation als Schutzmechanismus.** Der Ermüdungsterm der Mimose verhindert Stellgliedverschleiß durch Dauerschalten. Technisch entspricht dies einer *minimalen Schaltfrequenzbegrenzung* (Dwell Time), die in sicherheitskritischen Anwendungen (Schaltschütze, Ventile) unbedingt implementiert werden sollte.

5.10 Klasse VII – Der Tintenfisch: Multivariable adaptive Regelung

5.10.1 Biologisches Vorbild: Tintenfisch (*Octopus vulgaris*)

Der Tintenfisch ist eines der intelligentesten Wirbellosen und ein außergewöhnliches Beispiel für dezentral-distribuierte Intelligenz kombiniert mit adaptiver Umgebungsanpassung. Neun Zehntel seiner Neuronen befinden sich in seinen acht Armen – jeder Arm hat ein eigenes “Gehirn” und kann autonom handeln, wird aber durch das zentrale Gehirn koordiniert.

Regelungstechnische Charakteristika.

- **Mehrgrößenregelung (MIMO):** Acht Arme, jeder mit hunderten von Saugnäpfen, werden als ein koordiniertes System gesteuert. Dies entspricht einem MIMO-System mit $n \gg 1$ Eingangs- und Ausgangsvariablen.

- **Farbanpassung:** Chromatophoren (Farbzellen) werden durch ein dezentrales Regelungsnetz gesteuert, das innerhalb von Millisekunden Farbmuster auf der Haut erzeugt. Jede Zelle ist ein lokaler Akteur mit eigenem Regelkreis; die Koordination erfolgt durch neuronale Muster.
- **Adaptive Steifigkeit der Arme:** Je nach Aufgabe (Greifen, Schwimmen, Abtasten) passen die Arme ihre mechanische Steifigkeit an. Dies entspricht *impedanzgeregelten* Roboterarmen.
- **Lernen und Gedächtnis:** Tintenfische können Konditionierungsaufgaben lösen (klassische und operante Konditionierung), was auf einen adaptiven Lernmechanismus im zentralen Gangliensystem hindeutet.

Biologische Regelklasse 7 (Tintenfisch – Adaptive MIMO-Regelung mit verteilter Intelligenz). Die Tintenfisch-Klasse umfasst alle Regelkreise mit:

- Mehrgrößenstruktur: $\mathbf{C}(s) \in \mathbb{R}^{m \times p}$, $\mathbf{G}(s) \in \mathbb{R}^{p \times m}$.
- Dezentrale lokale Regler auf Aktorebene und einem zentralen Koordinationsregler.
- Adaptiver Parameteranpassung: $K_{ij}(t) = K_{ij,0} + \Delta K_{ij}(t)$.
- Signalflussgraph: Vollständig verbundener Multigraph (jeder Zustand beeinflusst jeden anderen Zustand) – die dichteste Klasse unter allen biologischen Äquivalenzklassen.

5.10.2 Graphenstruktur der Tintenfisch-Klasse

Der Tintenfisch-Regelkreis erzeugt die dichteste Adjazenzmatrix aller biologischen Klassen – annähernd die vollständige Matrix:

$$\mathbf{M}_{\text{Tintenfisch}} \approx \mathbf{J}_n = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{pmatrix}$$

Der Subgraph Algorithmus erkennt die Tintenfisch-Klasse daran, dass ihr Signalflussgraph *alle anderen biologischen Klassen als Subgraph enthält*: $\mathbf{M}_{\text{Seestern}} \subseteq \mathbf{M}_{\text{Kleinhirn}} \subseteq \mathbf{M}_{\text{Tintenfisch}}$ (vgl. Subgraph-Hierarchie Kapitel 5.11).

5.10.3 Abgeleitete Entwurfsstrategien

1. **Entkopplungsregler.** Ein vollständig verbundenes MIMO-System erfordert einen Entkopplungsregler, der die Quereinflüsse zwischen den Kanälen kompensiert. Die Tintenfisch-Klasse signalisiert daher in PyLab: “Entkopplung erforderlich!”
2. **Impedanzregelung für Roboterarme.** Die adaptive Steifigkeit der Tintenfischarme ist das biologische Vorbild für impedanzgeregelte Manipulatoren. Die gewünschte Impedanz $Z_{\text{ref}}(s) = Ms^2 + Ds + K$ wird als Referenz- Übertragungsverhalten vorgegeben und der Regler so ausgelegt, dass er dieses Verhalten gegenüber äußeren Kräften durchsetzt.

3. **Modellprädiktive MIMO-Regelung (MIMO-MPC).** Der Koordinationsmechanismus des zentralen Tintenfischgehirns entspricht einer modellprädiktiven Mehrgrößenregelung: Auf Basis eines internen MIMO-Modells werden optimale Eingaben für alle Kanäle gleichzeitig berechnet.

5.10.4 Erweiterungspotenzial: MIMO-Erweiterung von PyLab

Hypothese 6 (Tintenfisch-Erweiterung). Die Identifikation eines Regelkreises als Tintenfisch-Klasse motiviert die Erweiterung des Subgraph Algorithmus auf tensorproduktbasierte Graphen (vgl. Kapitel 9.2). Die Tintenfisch-Klasse ist der natürliche Startpunkt für die MIMO-Erweiterung von PyLab:

- Erweiterung der **TransferFunction**-Klasse auf Übertragungsmatrizen.
- Erweiterung des Subgraph Algorithmus auf blockstrukturierte Adjazenzmatrizen.
- Automatische Synthese von Entkopplungsreglern bei erkannter Tintenfisch-Topologie.

5.11 Hierarchie der biologischen Äquivalenzklassen

Die sieben biologischen Äquivalenzklassen bilden eine Hierarchie, die durch die Subgraph-Relation geordnet ist. Eine Klasse $\mathcal{K}_\alpha$ ist *Unterklasse* von $\mathcal{K}_\beta$, wenn jeder Regelkreis in $\mathcal{K}_\alpha$ graphenstrukturell in einem Regelkreis der Klasse $\mathcal{K}_\beta$ enthalten ist:

Satz 2 (Subgraph-Hierarchie der biologischen Klassen). Für die biologischen Äquivalenzklassen gilt folgende Subgraph-Ordnung:

$$\mathcal{K}_{\text{Seestern}} \subsetneq \mathcal{K}_{\text{Pupille}} \subsetneq \mathcal{K}_{\text{Kleinhirn}} \subsetneq \mathcal{K}_{\text{Herz}} \subsetneq \mathcal{K}_{\text{Albatros}} \subsetneq \mathcal{K}_{\text{Tintenfisch}}$$

Die Mimosen-Klasse steht strukturell zwischen Seestern und Pupille, ist jedoch nicht mit den übrigen Klassen vergleichbar (nichtlinear):

$$\mathcal{K}_{\text{Seestern}} \subsetneq \mathcal{K}_{\text{Mimose}} \not\subseteq \mathcal{K}_{\text{Pupille}}$$

Beweisskizze. – **Seestern $\subsetneq$ Pupille:** Der Integralknoten der Pupillen-Klasse (Selbstschleife $M_{11} = 1$) ist nicht in der Seestern-Adjazenzmatrix enthalten. Jede Seestern-Adjazenzmatrix ist jedoch (nach Padding) ein Subgraph der Pupillen-Matrix.

- **Pupille $\subsetneq$ Kleinhirn:** Der Vorsteuerblock der Kleinhirn-Klasse fügt zusätzliche Knoten hinzu, enthält aber die vollständige Pupillen-Topologie als Subgraph.
- **Kleinhirn $\subsetneq$ Herz:** Der Kaskadenblock der Herz-Klasse enthält zwei verschachtelte Schleifen; die Kleinhirn-Topologie ist als innere Schleife enthalten.
- **Herz $\subsetneq$ Albatros:** Der Störknoten der Albatros-Klasse erweitert den Herz-Graphen.
- **Albatros $\subsetneq$ Tintenfisch:** Der vollständig verbundene Tintenfisch-Graph enthält alle dünneren Graphen als Subgraph.

□

Kl.	Vorbild	Reglertyp	Graphdichte	Nichtlinear	Funktion
I	Seestern	P	gering	nein	Redundanz
II	Pupille	PI + Tot	mittel	nein	Anti-Windup, Log-Sca- -ling
III	Kleinhirn	PID + FF	mittel	nein	ILC, Lernregler
IV	Herz	Kaskade	mittel	nein	PWM-Takt, Kaskaden- schutz
V	Albatros	LQR + FF	hoch	nein	MPC, Störaufschaltung
VI	Mimose	2-Punkt	gering	ja	Grenzzyklus, Habitua- tion
VII	Tintenfisch	MIMO	sehr hoch	nein	Entkopplung, Impe- danz

Tabelle 4: Übersicht der sieben biologischen Äquivalenzklassen

5.12 Klassifikationsalgorithmus

5.12.1 Referenztopologien

Für jede biologische Äquivalenzklasse wird eine *Referenztopologie* $\mathbf{R}_\beta$ in der Datenbank hinterlegt. Die Referenztopologien werden als binäre Adjazenzmatrizen minimaler Dimension $n_{\min} \times n_{\min}$ gespeichert, wobei $n_{\min}$ die Mindestordnung des jeweiligen Regelkreistyps ist:

Klasse	Biologisches Vorbild	$n_{\min}$
I	Seestern	2
II	Pupille	3
III	Kleinhirn	4
IV	Herz	5
V	Albatros	4
VI	Mimose	2 (linearisiert)
VII	Tintenfisch	≥ 4

Tabelle 5: Mindestordnungen der biologischen Referenztopologien

5.12.2 Klassifikationsablauf

Die Klassifikation eines unbekannten Regelkreises $\mathcal{L}$ erfolgt durch sequentiellen Vergleich mit allen Referenztopologien, beginnend bei der spezifischsten Klasse (Tintenfisch):

1. Berechne $\mathbf{M}_\mathcal{L}$ durch den Pipeline-Schritt aus Kapitel 4.
2. Vergleiche $\mathbf{M}_\mathcal{L}$ mit $\mathbf{R}_{\text{Tintenfisch}}$: Ist $\mathbf{R}_{\text{Tintenfisch}} \subseteq \mathbf{M}_\mathcal{L}$? Falls ja: Klasse VII.
3. Sonst vergleiche mit $\mathbf{R}_{\text{Albatros}}$: Klasse V, falls Subgraph-Relation gilt.
4. Fortfahren absteigend durch die Hierarchie bis zur Seestern-Klasse.

5. Parallelprüfung auf Mimosen-Klasse (Nichtlinearitätstest: Prüfe ob die Regler-Übertragungsfunktion keine rationalen Koeffizienten hat).
6. Gibt keine Klasse eine positive Entscheidung, gilt: “Unklassifiziert”.

5.12.3 Implementierung: bio_classify.py

Das vorgeschlagene Modul `bio_classify.py` stellt die folgende öffentliche API bereit:

Listing 5: Öffentliche API des biologischen Klassifikationsmoduls

```

1 from pylabb.control.bio_classify import (
2     BioClass,                # Enum der sieben biologischen
3     Klassen
4     classify_loop,           # Regelkreis → BioClass + Konfidenz
5     BioClassResult,         # Datenklasse für das
6     Klassifikationsergebnis
7     get_extensions,         # BioClass → Liste empfohlener
8     Erweiterungen
9 )
10
11 # Beispielanwendung:
12 result = classify_loop(plant=G, controller=C)
13 print(result.bio_class)      # z.B. BioClass.KLEINHIRN
14 print(result.confidence)     # z.B. 0.87
15 print(result.subgraph_chain) # [Seestern, Pupille, Kleinhirn]
16 for ext in get_extensions(result.bio_class):
17     print(ext.name, ext.description)

```

Die Datenklasse `BioClassResult` enthält neben der Klassenzuweisung auch die vollständige Subgraph-Kette (welche Klassen als Subgraph enthalten sind) und eine Konfidenzschätzung, die auf der prozentualen Übereinstimmung der Adjazenzmatrix mit der Referenztopologie basiert.

5.13 Anwendungsbeispiele der biologischen Klassifikation

5.13.1 Beispiel 1: Temperaturregelung einer Heizung (Klasse II, Pupille)

Beispiel 2. Ein Heizkörper-Temperaturregler mit der Strecke:

$$G(s) = \frac{0,8}{(10s + 1)(30s + 1)} e^{-5s}$$

und einem PI-Regler $C(s) = 2 \left(1 + \frac{1}{25s}\right)$ wird in PyLab klassifiziert.

Ergebnis der Klassifikation: Die Adjazenzmatrix zeigt den charakteristischen Selbstschleifenknoten des Integralzustands. Der Subgraph Algorithmus erkennt $\mathbf{R}_{\text{Pupille}} \subseteq \mathbf{M}_{\mathcal{L}}$.

Klasse: II (Pupille) – Konfidenz: 94 %.

Abgeleitete Entwurfsempfehlung:

- Anti-Windup implementieren (Regler sättigt bei $u_{\max} = 100\%$ Heizleistung).
- Totzeitkompensation: Smith-Prädiktor mit $\hat{T}_t = 5\text{ s}$.
- Nichtlineares Feedforward: Heizkurve für Vorlauftemperatur in Abhängigkeit der Außentemperatur.

5.13.2 Beispiel 2: Roboterarm-Positionsregelung (Klasse III, Kleinhirn)

Beispiel 3. Ein Roboterarm mit einer Gelenkstrecke:

$$G(s) = \frac{K}{s(Js + b)}$$

(Doppelintegrator mit Dämpfung) und einem PID-Regler mit Vorsteuerung $U_{\text{FF}} = J_{\text{nom}} \cdot \ddot{q}_{\text{soll}}$ wird klassifiziert.

Ergebnis der Klassifikation: Die Adjazenzmatrix enthält den parallelen Vorwärtspfad-Knoten (Feedforward-Zweig) als separaten Block. Der Subgraph Algorithmus erkennt: $\mathbf{R}_{\text{Kleinhirn}} \subseteq \mathbf{M}_{\mathcal{L}}$.

Klasse: III (Kleinhirn) – Konfidenz: 89 %.

Abgeleitete Entwurfsempfehlung:

- ILC-Modul aktivieren: Bei repetitiver Trajektorie (z. B. Pick-and-Place-Roboter) lernt der Vorsteuerterm iterativ.
- Stör-Beobachter für Reibmoment und Lastmoment.
- `genmictory` erzeugt automatisch einen MicroPython-ILC-Loop für ESP32 mit anpassbarer Lernrate γ .

5.13.3 Beispiel 3: Windkraftanlage (Klasse V, Albatros)

Beispiel 4. Eine Windkraftanlage mit Pitchwinkelregelung:

$$G(s) = \frac{K_w}{(T_w s + 1)^2}, \quad d(t) = v_{\text{wind}}(t) \text{ (messbar)}$$

und einem LQR-Regler mit Windgeschwindigkeits-Aufschaltung: $u = -\mathbf{k}^T \mathbf{x} + f_{\text{FF}} \cdot d$.

Ergebnis der Klassifikation: Der Störeingangsknoten (Wind) ist in der Adjazenzmatrix als vollständig verbundene Zusatzspalte erkennbar. Der Subgraph Algorithmus erkennt: $\mathbf{R}_{\text{Albatros}} \subseteq \mathbf{M}_{\mathcal{L}}$.

Klasse: V (Albatros) – Konfidenz: 91 %.

Abgeleitete Entwurfsempfehlung:

- MPC-Erweiterung: Optimierung des Pitchwinkels über Horizont $H = 10 \text{ s}$ unter Ausnutzung der gemessenen Windvorhersage.
- Resonanzfilter für Blattfrequenz (1P, 3P) zur Reduktion von Ermüdungslasten.
- Riccati-Gleichung automatisch lösen und $\mathbf{k}$ in PyLab anzeigen.

5.14 Zusammenfassung und Ausblick

Dieses Kapitel hat gezeigt, dass die graphenstrukturelle Äquivalenzprüfung von Regelkreisen – die Kernfunktionalität von PyLab – als Grundlage für eine semantisch reichere Klassifikation nach biologischen Vorbildern dienen kann. Die sieben eingeführten biologischen Äquivalenzklassen spannen einen Raum auf, der vom einfachsten dezentralen Proportionalregler (Seestern) bis zur adaptiven MIMO-Regelung (Tintenfisch) reicht und dabei folgende Dimensionen abdeckt:

- **Komplexität der Regelstruktur** ($P \rightarrow PI \rightarrow PID \rightarrow \text{Kaskade} \rightarrow \text{LQR} \rightarrow \text{MIMO}$).
- **Grad der Nichtlinearität** (linear $\rightarrow$ totzeitbehaftet $\rightarrow$ hysteresisch $\rightarrow$ schaltend).
- **Adaptivität und Lernfähigkeit** (statisch $\rightarrow$ gainscheduled $\rightarrow$ iterativ lernend $\rightarrow$ modellprädiktiv).
- **Verteilung der Intelligenz** (zentral $\rightarrow$ kaskadiert $\rightarrow$ dezentral-verteilt).

Die biologische Klassifikation ist nicht nur ein akademisches Ordnungsprinzip, sondern ein praktisches Werkzeug: Sie ordnet jedem identifizierten Regelkreis eine Liste *konkreter Entwurfsempfehlungen und Erweiterungsmöglichkeiten* zu, die aus dem Verhalten des biologischen Vorbildes abgeleitet wurden.

6 GUI-Integration

6.1 Verifikations-Dock

Ab Version 2.0.0 enthält das PyLab-Hauptfenster ein *Regelkreis-Verifikations-Dock* (rechtes Seitenpanel). Es ist über das Menü **Verifikation $\rightarrow$ Regelkreis-Äquivalenz prüfen** oder die Taste **F7** erreichbar.

Das Dock besteht aus folgenden Elementen:

- **Eingabe-Panels A und B:** Jeweils vier Eingabefelder für Zähler und Nenner von Strecke $G(s)$ und Regler $C(s)$.
- **Entscheidungs-Badge:** Ein farbiges Label zeigt das Ergebnis an (grün: äquivalent, blau: A enthält B, lila: B enthält A, orange: verschieden).
- **Adjazenzmatrix-Ansichten:** Zwei Tabellen zeigen die binären Matrizen; Einträge mit dem Wert 1 sind grün hinterlegt.
- **Zusammenfassung:** Textuelles Feld mit der lesbaren Erklärung.
- **„Aktuellen Regelkreis als A übernehmen:** Synchronisiert Slot A mit dem System-Editor.

6.2 Automatische Synchronisation

Bei jeder Änderung im System-Editor (Strecke oder Regler) werden die Werte automatisch in Slot A des Verifikations-Docks übertragen. Dies ermöglicht einen nahtlosen Arbeitsfluss: Der Nutzer entwirft in der gewohnten Oberfläche und trägt einen Vergleichsregelkreis direkt im Dock ein.

6.3 Fehlerbehandlung in der GUI

Das Widget behandelt drei Fehlerklassen:

- **ValueError:** Ungültige Koeffizienten $\rightarrow$ Warnungsdialog
- **ImportError:** subgraph-Paket nicht installiert $\rightarrow$ Fehlerdialog mit Installationshinweis
- Sonstige Ausnahmen $\rightarrow$ Fehlerdialog mit Stack-Trace

7 Analyse

7.1 Zeitkomplexität

Satz 3 (Laufzeit). Die Verifikation zweier Regelkreise der Ordnung n hat eine Zeitkomplexität von $O(n^3)$.

Beweis. Die einzelnen Schritte:

1. **Offener Kreis:** Polynommultiplikation zweier Polynome der Grad $\leq n$: $O(n^2)$.
2. **Zustandsraum / Adjazenzmatrix:** `tf2ss` und Schwellwertvergleich der $n \times n$ -Matrix: $O(n^2)$.
3. **Subgraph Algorithmus:** Signaturberechnung $O(n^2)$, n Rotationen je mit LCS-Berechnung [11] $O(n^2)$: Gesamtkomplexität $O(n^3)$.

Damit dominiert Schritt 3: $O(n^3)$. □

7.2 Speicherkomplexität

Der Speicherbedarf beträgt $O(n^2)$ für die beiden Adjazenzmatrizen und $O(n)$ für die Signatur-Arrays.

7.3 Korrektheit

Satz 4 (Korrektheit). Der Subgraph Algorithmus erkennt eine Subgraph-Beziehung genau dann korrekt, wenn die Graphen unter zyklischer Rotation verglichen werden.

Beweis. Die zyklische Rotation entspricht einer Umnummerierung der Zustandsvariablen, die die Graphstruktur invariant lässt. Die Injektivität der Signaturabbildung (Lemma 1) garantiert, dass verschiedene Spaltenmuster verschiedene Signaturen erhalten und keine Fehlerkennungen auftreten. Die LCS-Prüfung ist korrekt, da sie exakt die gemeinsamen Muster beider Signaturreihen identifiziert. □

7.4 Adjazenzlisten-Variante

Für dichte Graphen ($m \approx n^2$) bietet das Modul eine alternative Implementation über Adjazenzlisten (`use_adjacency_list=True`), die den Speicherbedarf auf $O(n + m)$ reduziert.

7.5 Testabdeckung

Das Verifikationsmodul ist mit einer Testabdeckung von **100 %** abgedeckt (562 Tests, gemessen mit `pytest` [17] und `coverage.py`, Version 2.0.0):

8 Anwendungsbeispiele

8.1 Beispiel 1: P-Regler vs. PI-Regler

Ein PT2-System wird mit einem P- und einem PI-Regler geregelt. Da der PI-Regler einen Integrator einführt, erhöht sich die Systemordnung.

Datei	Zeilen	Abgedeckt	Coverage
control/verification.py	70	70	100 %
gui/widgets/verification.widget.py	183	183	100 %
Gesamt (alle Module)	–	–	99 %

Tabelle 6: Testabdeckung der Verifikationskomponenten

Listing 6: Verifikation P- vs. PI-Regler

```

1 from pylabb.core.transfer_function import TransferFunction
2 from pylabb.control.verification import verify_loops
3
4 # PT2-Strecke:  $G(s) = 1 / (s^2 + 1.4s + 1)$ 
5 G = TransferFunction([1], [1, 1.4, 1], name="PT2")
6
7 # P-Regler:  $C_P(s) = 2$ 
8 C_P = TransferFunction([2], [1], name="P")
9 # PI-Regler:  $C_{PI}(s) = (2s + 1) / s$ 
10 C_PI = TransferFunction([2, 1], [1, 0], name="PI")
11
12 result = verify_loops(G, C_P, G, C_PI,
13                      label_A="P-Regelkreis",
14                      label_B="PI-Regelkreis")
15
16 print(result.decision)      # "keep_b"
17 print(result.b_contains_a)  # True
18 print(result.summary)
19 # Ausgabe: "Graph von P-Regelkreis ist in PI-Regelkreis enthalten
20 #           --
21 #           PI-Regelkreis ist die umfangreichere Struktur."

```

Die Adjazenzmatrix des P-Regelkreises ist kleiner (2×2 für PT2 + P-Regler = Ordnung 2), die des PI-Regelkreises ist größer (3×3 , da der PI-Regler die Ordnung um 1 erhöht). Nach dem Padding auf 3×3 erkennt der Subgraph Algorithmus, dass der P-Graph im PI-Graph enthalten ist.

8.2 Beispiel 2: Strukturell verschiedene Regelkreise

Zwei Systeme erster Ordnung mit unterschiedlichen Reglerstrukturen:

Listing 7: Verifikation strukturell verschiedener Regelkreise

```

1 # PT1-Strecke mit PD-Regler
2 G1 = TransferFunction([1], [1, 1])
3 C1 = TransferFunction([1, 2], [1])    # PD:  $C(s) = s + 2$ 
4
5 # PT2-Strecke mit P-Regler
6 G2 = TransferFunction([1], [1, 0.5, 1])
7 C2 = TransferFunction([3], [1])      # P:  $C(s) = 3$ 
8
9 result = verify_loops(G1, C1, G2, C2)

```



```

10 print(result.decision)          # "keep_both"
11 print(result.are_independent) # True

```

8.3 Beispiel 3: Äquivalente Regelkreise

Zwei verschiedene Darstellungen desselben offenen Kreises:

Listing 8: Äquivalenznachweis zweier Regelkreise

```

1 # Darstellung 1: Direkte Reihenschaltung
2 G_a = TransferFunction([1], [1, 3, 2])
3 C_a = TransferFunction([5], [1])
4
5 # Darstellung 2: Factored
6 G_b = TransferFunction([1], [1, 1])
7 C_b = TransferFunction([5], [1, 2])
8
9 result = verify_loops(G_a, C_a, G_b, C_b)
10 print(result.are_equivalent) # True

```

9 Zusammenfassung

Diese Arbeit hat PyLab 2.0.0 vorgestellt – ein umfassendes Python-Framework für Regelungstechnik, das algebraische Systemmodellierung, Simulation, Visualisierung und MicroPython-Codegenerierung in einem einheitlichen Paket vereint.

Der Schwerpunkt lag auf dem neuen **Verifikationsmodul**, das den Subgraph Algorithmus auf die Regelungstechnik überträgt. Die Kernidee – einen Regelkreis in einen Signalflussgraphen zu überführen und dessen Adjazenzmatrix zyklisch zu vergleichen – liefert ein effizienz- und korrektkeitsnachweisbares Verfahren mit Zeitkomplexität $O(n^3)$ und vollständiger Testabdeckung (100%).

9.1 Wissenschaftlicher Beitrag

- Übertragung des Subgraph Algorithmus von der Programmverifikation auf die Regelkreisverifikation.
- Formale Definition des Signalflussgraphen eines Regelkreises über die Observable Canonical Form.
- Nachweis der Korrektheit und $O(n^3)$ -Laufzeit für den Verifikationsprozess.
- Integration in eine vollständige, GUI-gestützte Regelungstechnik-Entwicklungsumgebung.

9.2 Ausblick

Die in dieser Arbeit vorgestellten Ergebnisse bilden ein solides Fundament für eine Vielzahl weiterführender Forschungs- und Entwicklungsvorhaben. Der folgende Ausblick gliedert sich in sechs thematische Schwerpunkte: (i) das neue Paket **genmicropy** als zentrale Innovation für universelle MicroPython-Codegenerierung, (ii) tiefgehende Erweiterungen des

Verifikationsmoduls, (iii) algorithmische Skalierbarkeit und Parallelisierung, (iv) Weiterentwicklung der grafischen Benutzeroberfläche, (v) Hardware-in-the-Loop-Anbindung sowie (vi) wissenschaftliche und didaktische Perspektiven.

(i) **genmicroscopy** – Universelle MicroPython-Codegenerierung

Eine der bedeutendsten geplanten Erweiterungen des PyLab-Ökosystems ist das eigenständige Paket **genmicroscopy** (*Generate MicroPython*). Es soll als dediziertes Codegenerierungsmodul für *alle* verbreiteten MicroPython-Derivate [19] und -Geräte dienen und dabei den Prozess vom eingeregelter Modell bis zum deploymentfähigen Embedded-Code vollständig automatisieren.

Motivation. Aktuelle Codegenerierungs-Workflows für eingebettete Regler erfordern erheblichen manuellen Aufwand: Regelparameter werden händisch übersetzt, Plattformunterschiede (Registerlayout, Timer-Auflösung, Float-Unterstützung, verfügbare Kommunikationsperipherie) müssen individuell berücksichtigt werden, und die Einhaltung von Coding-Standards liegt vollständig beim Entwickler. **genmicroscopy** adressiert genau diese Lücke, indem es die in PyLab berechneten Reglerparameter (K_p , K_i , K_d , Abtastzeit T_s , Diskretisierungsverfahren) direkt in plattformspezifischen, sofort einsetzbaren MicroPython-Quellcode übersetzt.

Unterstützte Plattformen und Derivate. Das Paket soll von Beginn an eine breite Gerätebasis abdecken. Geplant sind Backends für:

- **ESP32 / ESP32-S2 / ESP32-S3 / ESP32-C3** (Espressif) – die am weitesten verbreitete MicroPython-Plattform mit WLAN, Bluetooth und umfangreichen PWM-/ADC-Peripherien.
- **RP2040 / RP2350** (Raspberry Pi Pico, Pico 2) – dual-core ARM Cortex-M0+/M33 mit PIO-Kooprozessor für präzises Timing; besonders relevant für Regelanwendungen mit engen Echtzeitanforderungen.
- **STM32** (STM32F4, STM32F7, STM32H7-Reihen) – leistungsstarke Cortex-M4/M7-Mikrocontroller mit Hardware-FPU, dediziert für hochfrequente Regelschleifen ($f_s > 10$ kHz).
- **SAMD21 / SAMD51** (Adafruit Circuit Python / MicroPython-forks) – weit verbreitet in der Maker-Community und im Bildungsbereich.
- **nRF52840** (Nordic Semiconductor) – BLE-fähige Plattform für drahtlose Regelanwendungen.
- **MIMXRT** (NXP i.MX RT Crossover MCUs) – für Industrie-nahe Anwendungen mit höherem Rechenanspruch.
- **Generisches MicroPython-Backend** – plattformagnostische Ausgabe, die auf jedem Standard-MicroPython-Port lauffähig ist.

Die Architektur von **genmicroscopy** ist bewusst erweiterbar gehalten: Neue Plattformen können durch Implementierung eines **BackendBase**-Interface ergänzt werden, ohne bestehenden Code zu modifizieren (Open/Closed Principle, OCP).

Code-Qualitätsprinzipien: MISRA, Clean Code und KISS. Ein zentrales Alleinstellungsmerkmal von `genmicrocopy` ist die konsequente Einhaltung etablierter Qualitätsstandards im *generierten* Code – nicht nur im Generator-Quellcode selbst. Drei Prinzipien stehen im Vordergrund:

MISRA-C/MicroPython-Richtlinien. MISRA C [20] (*Motor Industry Software Reliability Association*) definiert einen sicheren Teilsatz der Programmiersprache C für sicherheitskritische eingebettete Systeme. Da MicroPython letztlich in C implementiert ist und der generierte Python-Code direkt auf Embedded-Hardware läuft, werden analoge Prinzipien auf die MicroPython-Ebene übertragen: keine impliziten Typkonversionen ohne expliziten Cast, keine nicht initialisierten Variablen, strenge Einschränkung von Gleitkomma-Operationen auf dokumentierte Verwendungsstellen, vollständige Zweigabdeckung in allen Entscheidungsstrukturen, und Vermeidung rekursiver Aufrufe in zeitkritischen Pfaden. Jede generierte Datei enthält einen MISRA-Compliance-Header, der die Abweichungen bewusst dokumentiert.

Clean Code (R. C. Martin [21]). Generierter Code ist in erster Linie für Menschen lesbar – er wird debuggt, angepasst und weiterentwickelt. `genmicrocopy` erzeugt daher sprechende Bezeichner (`pid_compute`, `setpoint`, `error_prev`), konsistente Einrückung, Docstrings nach NumPy-Convention für alle öffentlichen Funktionen sowie Inline-Kommentare, die die regelungstechnische Bedeutung einzelner Berechnungsschritte erläutern (z.B. `# Trapezoidal integration of integral term`). Magische Zahlen werden grundsätzlich in benannte Konstanten ausgelagert; Seiteneffekte werden minimiert, indem der Reglerzustand ausschließlich über ein explizit übergebenes `state`-Dictionary geführt wird.

KISS (*Keep It Simple, Stupid*). Eingebettete Regler sollen so einfach wie möglich gehalten werden. `genmicrocopy` vermeidet unnötige Abstraktionsebenen im generierten Code: Statt einer vollständigen objektorientierten Klassenhierarchie erzeugt das Standard-Backend eine einzelne, selbst-dokumentierende Funktion `pid_step(state, setpoint, measurement)` sowie eine Initialisierungsfunktion `pid_init()`. Komplexere Objekt-Modelle sind optional über ein `--oo`-Flag aktivierbar. Diese klare Trennung von einfachem Standardpfad und optionaler Komplexität folgt dem Prinzip der minimalen Überraschung (*Principle of Least Astonishment*).

Geplante Ausgabeformate. Neben dem reinen `.py`-Quellcode soll `genmicrocopy` folgende Artefakte erzeugen können:

- **Standalone-Modul** (`pid_controller.py`) – direkt auf das Gerät kopierbar, keine externen Abhängigkeiten.
- **MicroPython-Frozen-Module** – vorkompiliert als `.mpy`-Bytecode für ressourcenbeschränkte Geräte.
- **C-Extension-Stub** – Vorlage für eine native C-Implementierung des Regelgesetzes, die über `micropython.native` oder `@micropython.viper` aufgerufen werden kann.
- **CMake/Make-Integration** – automatisch generierte Build-Skripte für die Einbettung in größere Firmware-Projekte.

- **Hardware-Abstraktions-Layer (HAL)** – plattformspezifischer Wrapper für ADC-Lesen, PWM-Ausgeben und Timer-Setup, sodass der Regelkern hardware-unabhängig bleibt.

Integration in PyLab. `genmicrocopy` wird als optionale Abhängigkeit in PyLab eingebunden: Das bestehende `codegen`-Modul delegiert an `genmicrocopy`, sobald es installiert ist, und fällt andernfalls auf die bestehende, vereinfachte Implementierung zurück. Die GUI erhält einen neuen *Code Generator*-Dialog, der Plattform, Ausgabeformat, Codestil (MISRA, Clean Code, minimal) sowie Diskretisierungsverfahren (Euler, Tustin/Bilinear, ZOH) interaktiv konfigurierbar macht. Ein integriertes Vorschaufenster zeigt den generierten Code unmittelbar an, bevor er exportiert wird.

(ii) Erweiterungen des Verifikationsmoduls

Diskrete Regelkreise (z-Domäne). Das derzeitige Verfahren operiert ausschließlich im kontinuierlichen Laplace-Bereich (*s*-Domäne). Eine natürliche Erweiterung ist die Unterstützung diskreter Regelkreise in der *z*-Domäne. Die Zustandsraum-Darstellung $\mathbf{x}[k+1] = A_d \mathbf{x}[k] + B_d u[k]$ liefert dieselbe Adjazenzmatrixstruktur; der Subgraph Algorithmus bleibt formal unverändert. Besondere Sorgfalt ist bei der Behandlung verschiedener Diskretisierungsverfahren (Euler vorwärts/rückwärts, Tustin, ZOH [6]) erforderlich, da diese die Pol-Nullstellen-Lage und damit die Graphstruktur beeinflussen.

Einbeziehung von Stabilitätseigenschaften. Bislang prüft das Verfahren ausschließlich die topologische Struktur des Signalflussgraphen. Eine wertvolle Erweiterung wäre die Annotation der Graphkanten mit Stabilitätsinformation: Pole, die in der linken Halbebene (kontinuierlich) bzw. innerhalb des Einheitskreises (diskret) liegen, erhalten ein positives Stabilitätslabel; instabile Pole ein negatives. Das erweiterte Verifikationskriterium würde dann zusätzlich fordern, dass beide Regelkreise dieselbe *Stabilitätsklasse* aufweisen – ein erheblicher Mehrwert für sicherheitsrelevante Anwendungen.

Nichtlineare Regelkreise. Nichtlineare Systeme lassen sich durch Linearisierung um einen Arbeitspunkt näherungsweise in der Zustandsraumdarstellung beschreiben. Eine automatische Jacobi-Linearisierung, gestützt auf symbolische Differentiation mittels `sympy`, würde den Anwendungsbereich des Verifikationsmoduls erheblich erweitern. Langfristig ist eine vollständige Erweiterung auf hybride Zustandsautomaten denkbar, die auch schaltende und stückweise lineare Regelungsstrukturen erfassen.

Parametrische Empfindlichkeitsanalyse. Ein weiterführendes Ziel ist die *robuste* Verifikation: Statt zwei konkreter Parametersätze werden Parameterbereiche verglichen. Intervall-Arithmetik oder Monte-Carlo-basierte Methoden könnten für jeden Stichprobenpunkt eine Graphäquivalenzprüfung durchführen und so eine Wahrscheinlichkeitsaussage über die strukturelle Ähnlichkeit zweier Regelkreis-Familien liefern.

(iii) Algorithmische Skalierbarkeit und Parallelisierung

Für Systeme hoher Ordnung ($n > 100$) dominiert die zyklische Rotationsprüfung mit $O(n^3)$ die Laufzeit. Mehrere Optimierungsstrategien bieten sich an:

- **Parallelisierung der Rotationsprüfungen** mittels `multiprocessing` oder `concurrent.futures`: Da alle n Rotationen unabhängig voneinander sind, skaliert die Beschleunigung nahezu linear mit der Anzahl verfügbarer CPU-Kerne.
- **GPU-Beschleunigung** über `cupy` oder `torch`: Matrixmultiplikation und Frobenius-Normen lassen sich auf modernen GPUs um Größenordnungen schneller berechnen als auf der CPU.
- **Approximative Verfahren**: Für sehr große Systeme könnten randomisierte Sketching-Methoden (z. B. Johnson-Lindenstrauss [24]) eine sublineare Vorabprüfung ermöglichen, die exakte Prüfung auf vielversprechende Kandidaten beschränkt.
- **Sparse-Adjazenzmatrizen**: Regelkreise mit wenigen Kopplungen – typisch für modulare Industrieregler – führen auf schwach besetzte Matrizen. Eine konsequente Sparse-Darstellung (`scipy.sparse`) würde sowohl Speicher als auch Rechenzeit signifikant reduzieren.

(iv) Weiterentwicklung der grafischen Benutzeroberfläche

Die aktuelle PyQt6-GUI stellt einen funktionsfähigen Prototypen dar, bietet aber erhebliches Ausbaupotenzial:

- **Interaktiver Signalflussgraph-Editor**: Statt einer rein tabellarischen Eingabe der Übertragungsfunktionen sollte der Benutzer Blöcke und Verbindungen direkt auf einem Zeichenfeld platzieren können. Die graphenstrukturelle Darstellung würde die Verifikationsergebnisse unmittelbar visuell kommunizieren.
- **Live-Simulation**: Eine Echtzeit-Simulation des geschlossenen Kreises mit einstellbarer Führungsgröße direkt in der GUI würde den Workflow erheblich straffen.
- **Berichterstellung**: Automatische Generierung von PDF-Berichten (via $\text{\LaTeX}$ -Templates oder `reportlab`) mit Bodediagramm, Sprungantwort, Verifikationsergebnis und generiertem Code.
- **Web-Frontend**: Eine auf `PyScript` oder `panel/holoviews` basierende Browser-Variante würde PyLab ohne lokale Installation zugänglich machen – wertvoll für den Einsatz in Lehrveranstaltungen.
- **Dark-Mode und Barrierefreiheit**: Vollständige Unterstützung des System-Themes sowie gesteigerte Zugänglichkeit gemäß WCAG 2.1.

(v) Hardware-in-the-Loop (HiL) und Deployment-Workflow

Mittelfristig soll PyLab einen durchgängigen Workflow von der Modellierung bis zum physisch laufenden Regler unterstützen:

1. **Modellierung und Auslegung** im PyLab-Framework (bereits vorhanden).
2. **Verifikation** der Regelkreisstruktur mittels Subgraph Algorithmus (bereits vorhanden).
3. **Codegenerierung** für die Zielhardware via `genmicrocopy` (geplant).

4. **Automatisches Deployment** auf das angeschlossene MicroPython-Gerät per `mpremote` oder `rshell` – direkt aus der GUI heraus.
5. **Hardware-in-the-Loop-Test**: PyLab sendet Sollwertsequenzen an das Gerät und liest Istwerte zurück; die gemessene Sprungantwort wird mit der simulierten verglichen und auf Abweichungen geprüft.
6. **Automatische Regleroptimierung**: Auf Basis der HiL-Daten können Reglerparameter iterativ nachgeführt werden (z. B. via Nelder-Mead-Simplex [22] oder Bayes'scher Optimierung [23]), bis Soll- und Ist-Antwort hinreichend übereinstimmen.

Dieser geschlossene Kreislauf aus Simulation, Verifikation, Codegenerierung und Hardware-Validierung würde PyLab von einem akademischen Werkzeug zu einer vollständigen Embedded-Control-Entwicklungsumgebung aufwerten.

(vi) Wissenschaftliche und didaktische Perspektiven

Formale Verifikation. Der Subgraph Algorithmus wurde in dieser Arbeit durch Testabdeckung und informelle Beweise validiert. Eine weitergehende Zielsetzung ist der vollständig formale Korrektheitsbeweis, etwa im Beweisassistenten Coq [25] oder Isabelle/HOL [26]. Insbesondere die Korrektheit des Padding-Schemas für ungleich große Graphen sowie die Vollständigkeit (kein False Negative) bieten sich als Beweis-Lemmas an.

Erweiterung auf Mehrgrößensysteme (MIMO). Das derzeitige Verfahren ist auf Eingrößensysteme (SISO) ausgerichtet. Mehrgrößenregelkreise (MIMO) führen auf blockstrukturierte Zustandsraummatrizen; der Subgraph Algorithmus [12] müsste auf Tensorprodukt-Graphen verallgemeinert werden. Dies eröffnet einen eigenständigen Forschungsstrang an der Schnittstelle von Graphentheorie und Mehrgrößenregelungstechnik.

Einsatz in der Lehre. PyLab und `genmicropy` eignen sich hervorragend für den Einsatz in Lehrveranstaltungen zu Regelungstechnik und Eingebetteten Systemen. Konkret geplant sind:

- Jupyter-Notebook-basierte interaktive Übungen, in denen Studierende Regelkreise modellieren, verifizieren und direkt auf einem Raspberry Pi Pico beobachten können.
- Ein öffentlich zugängliches Docker-Image mit vollständig vorkonfigurierter PyLab-Umgebung.
- Begleitende Lehrvideos und ein strukturiertes Tutorial, das den Weg von der Übertragungsfunktion bis zum laufenden Embedded-Regler Schritt für Schritt demonstriert.

Veröffentlichung und Community. `genmicropy` soll als eigenständiges Open-Source-Paket auf PyPI veröffentlicht werden, um eine breite Community von Embedded-Entwicklern anzusprechen, die MicroPython einsetzen, aber bislang keine standardkonformen Codegeneratoren für Regler zur Verfügung hatten. Eine aktive Community würde die Plattformunterstützung organisch erweitern und QA-Tests auf realer Hardware beisteuern – ein Mehrwert, der durch rein akademische Entwicklung nicht erreichbar ist.

Zusammenfassend steht PyLab am Beginn eines Entwicklungspfades, der über das akademische Verifikationswerkzeug hinaus zu einer durchgängigen, normkonformen und hardware-nahen Regelungstechnik-Entwicklungsumgebung führt. Die Einführung von `genmicropy` markiert dabei den ersten konkreten Schritt in Richtung industrieller Praxistauglichkeit: Vom Regelungsmodell zum *deployment-fähigen, MISRA-orientierten, lesbaren und geräteübergreifenden* MicroPython-Code – vollständig automatisiert und in PyLab integriert.

Literatur

- [1] Stephan Epp. *Der Subgraph Algorithmus*, 7. Januar 2026. Verfügbar unter: <https://github.com/hjstephan86/subgraph>
- [2] Katsuhiko Ogata. *Modern Control Engineering*, 5. Auflage. Pearson, 2010.
- [3] Jan Lunze. *Regelungstechnik 1 – Systemtheoretische Grundlagen, Analyse und Entwurf einschleifiger Regelungen*, 10. Auflage. Springer Vieweg, 2016.
- [4] Otto Föllinger, Uwe Konigorski. *Regelungstechnik – Einführung in die Methoden und ihre Anwendung*, 12. Auflage. VDE Verlag, 2020.
- [5] Karl J. Åström, Tore Hägglund. *PID Controllers: Theory, Design, and Tuning*, 2. Auflage. Instrument Society of America, 1995.
- [6] Gene F. Franklin, J. David Powell, Abbas Emami-Naeini. *Feedback Control of Dynamic Systems*, 8. Auflage. Pearson, 2019.
- [7] John G. Ziegler, Nathaniel B. Nichols. *Optimum Settings for Automatic Controllers*. Transactions of the ASME, 64(8), 759–768, 1942.
- [8] G. H. Cohen, G. A. Coon. *Theoretical Consideration of Retarded Control*. Transactions of the ASME, 75, 827–834, 1953.
- [9] Samuel J. Mason. *Feedback Theory – Some Properties of Signal Flow Graphs*. Proceedings of the IRE, 41(9), 1144–1156, 1953.
- [10] Reinhard Diestel. *Graph Theory*, 5. Auflage. Springer, 2017.
- [11] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*, 4. Auflage. MIT Press, 2022.
- [12] Julian R. Ullmann. *An Algorithm for Subgraph Isomorphism*. Journal of the ACM, 23(1), 31–42, 1976.
- [13] Pauli Virtanen et al. *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*. Nature Methods, 17, 261–272, 2020.
- [14] Charles R. Harris et al. *Array Programming with NumPy*. Nature, 585, 357–362, 2020.
- [15] John D. Hunter. *Matplotlib: A 2D Graphics Environment*. Computing in Science & Engineering, 9(3), 90–95, 2007.
- [16] The Qt Company. *Qt 6 Documentation*. <https://doc.qt.io/qt-6/>, 2024.

- [17] Holger Krekel et al. *pytest – A Mature Full-featured Python Testing Tool*. <https://pytest.org>, 2024.
- [18] The MathWorks, Inc. *MATLAB – The Language of Technical Computing*. <https://www.mathworks.com/products/matlab.html>, 2024.
- [19] Damien P. George et al. *MicroPython – Python for Microcontrollers*. <https://micropython.org>, 2025.
- [20] MISRA Consortium. *MISRA C:2012 – Guidelines for the Use of the C Language in Critical Systems*, 3. Ausgabe. MIRA Limited, 2019.
- [21] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [22] John A. Nelder, Roger Mead. *A Simplex Method for Function Minimization*. The Computer Journal, 7(4), 308–313, 1965.
- [23] Jonas Mockus. *Bayesian Approach to Global Optimization – Theory and Applications*. Kluwer Academic Publishers, 1989.
- [24] William B. Johnson, Joram Lindenstrauss. *Extensions of Lipschitz Mappings into a Hilbert Space*. Contemporary Mathematics, 26, 189–206, 1984.
- [25] The Coq Development Team. *The Coq Proof Assistant Reference Manual*. <https://coq.inria.fr>, 2025.
- [26] Tobias Nipkow, Lawrence C. Paulson, Markus Wenzel. *Isabelle/HOL – A Proof Assistant for Higher-Order Logic*. Springer, 2002.
- [27] J. A. Scott Kelso. *Dynamic Patterns: The Self-Organization of Brain and Behavior*. MIT Press, 1995.
- [28] Eric R. Kandel, James H. Schwartz, Thomas M. Jessell. *Principles of Neural Science*, 5. Auflage. McGraw-Hill, 2012.
- [29] John C. Doyle, Bruce A. Francis, Allen R. Tannenbaum. *Feedback Control Theory*. Macmillan, 1992.
- [30] Suguru Arimoto, Sadao Kawamura, Fumio Miyazaki. *Bettering Operation of Robots by Learning*. Journal of Robotic Systems, 1(2), 123–140, 1984.
- [31] Sigurd Skogestad, Ian Postlethwaite. *Multivariable Feedback Design: Analysis and Synthesis*. Wiley, 2005.
- [32] Gustav Fechner. *Elemente der Psychophysik*. Breitkopf und Härtel, Leipzig, 1860.